

Bachelor thesis

Study of regularization mechanisms in neural networks



Adam Maltoni

Higher Polytechnic School
Autonomous University of Madrid
C/ Francisco Tomás y Valiente nº 11

**UNIVERSIDAD AUTÓNOMA DE MADRID
ESCUELA POLITÉCNICA SUPERIOR**



Bachelor as Data Science and Engineering

BACHELOR THESIS

**Study of regularization mechanisms in neural
networks**

Quantification based on internal activation dynamics

**Author: Adam Maltoni
Advisor: Dr. Luis Fernando Lago Fernández**

October 2026

Some rights reserved

This work is licensed under a Creative Commons
Attribution-NonCommercial-ShareAlike 3.0 License.
<http://creativecommons.org/licenses/by-nc-sa/3.0/>

You are free to share (copy, distribute and transmit) and to modify the work
under the following conditions:

- You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).
- You may not use this work for commercial purposes.
- If you alter, transform, or build upon this work, you may distribute the resulting work only under the same or similar license to this one.

RIGHTS RESERVED

© October 2026 por UNIVERSIDAD AUTÓNOMA DE MADRID
Francisco Tomás y Valiente, nº 1
Madrid, 28049
Spain

Adam Maltoni

Study of regularization mechanisms in neural networks

Adam Maltoni

C\ Francisco Tomás y Valiente N^o 11

PRINTED IN SPAIN

*To all my loved ones for their unconditional support,
to my professors for their dedication and support
and to the scientific community for its inspiration.*

*Perfection is achieved, not when there is nothing more to add,
but when there is nothing left to take away.*

Antoine de Saint-Exupéry

PREFACE

This work arises from the motivation to understand what really happens in the layers of a neural network when we try to force its generalization capacity. In the usual practice of deep learning, we apply regularization techniques almost systematically; they work and improve evaluation metrics, but we often do so without a clear understanding of the internal effect they cause.

This «black box», at the moment, intrigues me more than obtaining high accuracies itself. My interest has shifted from pure efficiency towards a more fundamental question: what differentiates, at a structural level, a network that simply memorizes from one that manages to extract general patterns?

The goal is to observe the model from within and try to provide a more tangible description of its behavior. It does not seek to offer a definitive solution, but rather to approach the problem from a different perspective, and stop assuming the functioning of regularization techniques as a given.

Adam Maltoni

ACKNOWLEDGMENTS

I would like to express my gratitude to my parents and closest loved ones for their constant support throughout my life and this project.

Also to the scientific and university community, whose work and teachings throughout this entire stage have inspired and shaped me in many ways.

Specially, I want to thank Luis Lago for his accompaniment, dedication, and guidance throughout this entire work.

ABSTRACT

This work investigates how regularization techniques affect not only the final accuracy of neural networks, but also what happens «inside» them during learning. Instead of viewing the network as a black box, we analyzed the evolution of its internal activations using information theory metrics, such as Shannon entropy. Through experiments with CIFAR-10, SVHN, FashionMNIST, and CIFAR-100, we observed that the models that generalize best tend to lie in intermediate regions of internal dispersion of their activations. These results suggest that there is a relationship between the «geometry» of internal representations and the model's ability to avoid overfitting, offering a perspective that goes beyond accuracy alone for evaluating regularizers according to the problem.

KEYWORDS

neural networks, regularization, deep learning, internal activations, generalization gap

TABLE OF CONTENTS

1	Introduction	1
1.1	Context and motivation of the work	1
1.2	Problem to address	2
1.3	General and specific objectives of the work	2
1.4	Document structure	3
2	Theoretical Framework	5
2.1	Neural networks for <i>deep learning</i>	5
2.1.1	Concept	5
2.1.2	Training process	5
2.1.3	Application to images: Convolutional neural networks	6
2.2	The Problem of Overfitting and Generalization	7
2.3	Regularization Techniques	7
2.3.1	L_1 Penalty (Lasso)	8
2.3.2	L_2 Penalty (Ridge)	8
2.3.3	Dropout	8
2.3.4	<i>Early Stopping</i>	9
2.3.5	Gaussian Noise Injection (<i>Gaussian Noise</i>)	9
2.3.6	<i>Data Augmentation</i>	9
2.3.7	<i>Batch Normalization</i>	9
2.4	Measures for the dynamics of internal activations	10
2.4.1	Probability Distributions and Entropy	10
2.4.2	Activation Dispersion	11
3	State of the Art	13
3.1	Generalization in deep models	13
3.2	Empirical evaluation of regularization	13
3.3	Domain dependence and performance variability	14
3.4	Towards a «total» evaluation	15
4	Experimental Design and Methodology	17
4.1	Neural network architecture	17
4.2	Evaluation with multiple datasets	17
4.3	Hyperparameter search	18
4.4	Training pipeline	19
4.5	Activation extraction system	19
4.6	Activation quantification: metrics	20
4.6.1	Discretization and vectorization of internal states	20
4.6.2	Shannon entropy and activation dispersion	20
4.7	Main experiments	21
4.8	Computational environment and reproducibility	22
5	Results and Discussion	23

5.1 Accuracy Results: Hyperparameter Search (EXP-1)	23
5.2 Internal Activations Analysis	24
5.2.1 Correlation between Internal Metrics and Performance (EXP-2)	24
5.2.2 Temporal Evolution of Representations (EXP-3)	28
5.2.3 Robustness to Data Noise (EXP-4)	29
5.2.4 Robustness to Weight Noise and Flat Minima Analysis (EXP-5)	32
5.3 Discussion	36
6 Conclusions and Future Work	39
6.1 Conclusions	39
6.2 Future Work	40
6.2.1 Experimental extensions	40
6.2.2 Methodological lines	40
Bibliography	42
Appendices	43
A Additional activation histograms	45
A.1 CIFAR-10	45
A.2 SVHN	49
A.3 CIFAR-100	52
A.4 FashionMNIST	56
B Correlations between considered internal metrics	59
C Accuracy retention tables by data noise level	67
D Accuracy retention tables by weight noise level	73

LISTS

List of equations

2.1	Activation of the hidden layer l	5
2.2	Training process	5
2.3	Cross-Entropy	6
2.4	Convolution	6
2.5	Pooling	7
2.6	Definition of the cost function with L_1 regularization (Lasso).	8
2.7	Definition of the cost function with L_2 regularization (Ridge).	8
2.8	Definition of activation with Dropout.	8
2.9	Definition of Gaussian noise injection.	9
2.10	Definition of Gaussian noise injection on weights.	9
2.11	Standardization and affine transformation formulas in Batch Normalization.	10
2.12	Definition of Shannon entropy.	10
2.13	Definition of the standard deviation.	11

List of figures

5.1	Ranking of val_acc	23
5.2	Generalization gap ranking	24
5.3	Visualization of activation compression	25
5.4	Relationship between internal metrics and generalization	26
5.5	Temporal evolution of unique_pctg in fc1 by dataset	28
5.6	Accuracy degradation curves against data noise	29
5.7	Visualization of the robustness of three images to noise	30
5.8	Accuracy degradation due to weight noise in conv1 and conv3	33
5.9	Accuracy degradation due to weight noise in fc1	34
5.10	Comparison of accuracy retention by layer	36
5.11	Normalized multidimensional evaluation	37
A.1	Activation visualization - CIFAR10, BatchNorm, 1.0	45
A.2	Activation visualization - CIFAR10, DataAug, 3.0	46
A.3	Activation visualization - CIFAR10, Dropout, 0.6	47
A.4	Activation visualization - CIFAR10, L2, 0.01	48
A.5	Activation visualization - SVHN, BatchNorm, 1.0	49
A.6	Activation visualization - SVHN, Dropout, 0.6	50
A.7	Activation visualization - SVHN, L1, 0.01	51
A.8	Activation visualization - CIFAR100, BatchNorm, 1.0	52
A.9	Activation visualization - CIFAR100, DataAug, 3.0	53
A.10	Activation visualization - CIFAR100, Dropout, 0.6	54
A.11	Activation visualization - CIFAR100, L1, 0.01	55

A.12	Activation visualization - FashionMNIST, BatchNorm, 1.0	56
A.13	Activation visualization - FashionMNIST, Dropout, 0.6	57
A.14	Activation visualization - FashionMNIST, GaussianNoise, 0.2	58
B.1	Correlation between internal metrics on CIFAR-10	59
B.2	Internal metrics and accuracy on CIFAR-10	60
B.3	Correlation between internal metrics on SVHN	61
B.4	Internal metrics and accuracy on SVHN	62
B.5	Correlation between internal metrics on CIFAR-100	63
B.6	Internal metrics and accuracy on CIFAR-100	64
B.7	Correlation between internal metrics on Fashion-MNIST	65
B.8	Internal metrics and accuracy on Fashion-MNIST	66

List of tables

4.1	Architecture of the CNN used in all experiments. The output sizes correspond to $32 \times 32 \times 3$ inputs (CIFAR-10 and SVHN).	18
5.1	Activation metrics in f_{c1} for the optimal scenario of each method in the four evaluated datasets.	27
5.2	Retention of <i>accuracy</i> against Gaussian data noise ($\sigma=0.2$) by method in the four evaluated datasets.	31
5.3	Retention of <i>accuracy</i> against weight noise ($\sigma=0.15$) by layer and method in the four evaluated datasets.	35
C.1	<i>Accuracy</i> retention against Gaussian noise in data ($\sigma=0.05$) per method on the four evaluated datasets.	67
C.2	<i>Accuracy</i> retention against Gaussian noise in data ($\sigma=0.1$) per method on the four evaluated datasets.	68
C.3	<i>Accuracy</i> retention against Gaussian noise in data ($\sigma=0.2$) per method on the four evaluated datasets.	69
C.4	<i>Accuracy</i> retention against Gaussian noise in data ($\sigma=0.3$) per method on the four evaluated datasets.	70
C.5	<i>Accuracy</i> retention against Gaussian noise in data ($\sigma=0.4$) per method on the four evaluated datasets.	71
C.6	<i>Accuracy</i> retention against Gaussian noise in data ($\sigma=0.5$) per method on the four evaluated datasets.	72
D.1	<i>Accuracy</i> retention against Gaussian noise in weights ($\sigma=0.005$) per layer and method on the four evaluated datasets.	73
D.2	<i>Accuracy</i> retention against Gaussian noise in weights ($\sigma=0.01$) per layer and method on the four evaluated datasets.	74
D.3	<i>Accuracy</i> retention against Gaussian noise in weights ($\sigma=0.02$) per layer and method on the four evaluated datasets.	75
D.4	<i>Accuracy</i> retention against Gaussian noise in weights ($\sigma=0.03$) per layer and method on the four evaluated datasets.	76
D.5	<i>Accuracy</i> retention against Gaussian noise in weights ($\sigma=0.05$) per layer and method on the four evaluated datasets.	77

D.6	<i>Accuracy</i> retention against Gaussian noise in weights ($\sigma=0.075$) per layer and method on the four evaluated datasets.	78
D.7	<i>Accuracy</i> retention against Gaussian noise in weights ($\sigma=0.1$) per layer and method on the four evaluated datasets.	79
D.8	<i>Accuracy</i> retention against Gaussian noise in weights ($\sigma=0.15$) per layer and method on the four evaluated datasets.	80

INTRODUCTION

1.1 Context and motivation of the work

In the current technological context, in which data-driven prediction models have taken and are taking an increasingly important role in numerous sectors and use cases, a deep understanding of what governs their performance is required. This cannot be restricted to the created models merely being able to predict known data, but rather, to achieve real utility in a production context, it is also necessary that they are capable of «understanding» general patterns, that is, making adequate predictions based on common features learned during training (otherwise, simply storing a database with the training samples would be enough, and *machine learning* models would have no utility). Therefore, it would be desirable for our predictors to always be able to adjust based on the underlying structure of the data and not just the numbers they show. To give an example, let's take a small child who is learning to add: first they will learn that $1 + 1 = 2$, then that $1 + 2 = 3$, etc. The most likely outcome would be that, if we asked them what the result of $3 + 1$ is, they would not be able to give the answer initially, but that, after a little practice and more examples of additions, they would understand that what they have to do is «join the two amounts and count the resulting number of fingers».

This property is what we call «generalization capacity», and, commonly, it is taken as the *accuracy* of our model on the validation set, that is, its predictive performance on data that the algorithm has never seen before («how well the child adds»), assuming that these present the same shape as those with which it has been trained («the child still does not know how to add negative numbers»).

Returning to technicalities, neural networks seem to be the current leader among machine learning algorithms for capturing complex relationships in data. However, certain architectures such as deep neural networks (*Deep Neural Networks*) have such an amount of parameters that their representational capacity is far superior to the complexity of the data they model. It has even been empirically demonstrated that these networks have the ability to simply memorize an entire training dataset, achieving zero error even when the original data labels are replaced by pure random noise [1].

To prevent the network (or algorithm in general) from memorizing the data without «understanding» it, that is, to force it to look for more general patterns in the data, the standard used are regularization techniques, which introduce restrictions or penalties during training to «push» the model towards simpler and more generalizable solutions.

However, the way in which the effectiveness of a regularizing method on a dataset and with an algorithm is usually evaluated is, surprisingly, superficial. In the vast majority of practical cases, these techniques are applied treating the neural network as a «black box», in which the success or failure of a technique is determined almost exclusively by observing external performance metrics, such as accuracy on the validation set or a decrease in the loss function. While these metrics are certainly useful for evaluating the overall operation of the model, they do not offer us any information about what is happening inside the network, or why the regularization is working or not in a specific case, limiting us

to observing the final result and testing parameters empirically until finding a «favorable» configuration.

This external view of the results prevents us from deeply understanding the real mechanisms that govern the model's learning, and this is precisely the main motivation of this work: to contribute to the progress towards more interpretable *machine learning* systems that generalize better, starting by investigating how different regularization techniques alter the internal behavior of neural networks when training and predicting data, and trying to find a measure of what is «good regularization» beyond the observation of an empirical improvement in validation accuracy.

1.2 Problem to address

The fundamental derived problem is the absence of «standardized» metrics and methodologies to measure the internal («real») effect of regularization. As previously mentioned, current scientific literature validates these techniques based almost entirely on the reduction of error in test sets. This turns the selection and adjustment of regularization hyperparameters into a trial and error process, instead of a decision based on the behavior of the network, and without being aware of what is really being done.

Although we know the theoretical foundation of numerous methods, such as $L1$, $L2$ penalties or other techniques like *Dropout*, there are not many studies that try to prove how this translates into quantifiable changes in the weights and/or distribution of activations in the hidden layers throughout the training epochs. Thus, it has not been fully understood why the same technique can be highly effective in some datasets but even useless in others [2].

1.3 General and specific objectives of the work

Therefore, the **general objective** of the study lies in conducting an observable analysis on this «black box», quantifying and evaluating the impact of regularization techniques in deep neural networks through direct collection and analysis of internal activations and their distribution and dynamics.

To achieve this end, the following **specific objectives** have been established:

- (O1) Design a testing environment in which to experiment, using standard architectures and classic reference datasets (to be able to establish reproducible and objective comparisons between the different evaluated methods).
- (O2) Adapt the model training process to extract and store the temporal evolution (by epochs) both of the general performance metrics (*accuracy* and loss function in training and validation, depending on the regularization method and parameter used) and of the hidden layer activations.
- (O3) Compare the internal impact generated by different types of regularization strategies (example: $L1$ or $L2$, based on a mathematical penalty, versus *Dropout* or *Data augmentation*, based on turning off random connections and expanding the dataset through various transformations, respectively).
- (O4) Relate (empirically) the observed changes in internal representations with the final generalization capacity of the model.
- (O5) Analyze the evolution of the regularizing effect across the depth of the network, that is, studying the differences in impact of these techniques along the different layers of a network.
- (O6) Propose, based on the findings, observations and considerations regarding the con-

cept of «good» regularization, with the ambition of somewhat reducing the dependence on *accuracy* and choosing techniques and parameters in a more «explainable» way.

From these objectives, three **research questions** are derived that this work seeks to answer empirically:

- 1.– **(RQ1)** Is there a systematic relationship between internal activation metrics and the observed performance of the model?
- 2.– **(RQ2)** Is validation *accuracy* a sufficient criterion to evaluate the quality of regularization, or are there relevant dimensions that this indicator does not capture?
- 3.– **(RQ3)** Is there a universally superior regularization technique, or does the optimal choice depend on the properties of the problem?

The answers to these questions are presented in Chapter 6.1.

1.4 Document structure

To address the stated objectives in a systematic and rigorous manner, this report has been structured into six chapters, the contents of which are detailed below:

Chapter 2 establishes the fundamental theoretical framework that supports the work. It details the mathematical principles of deep learning, including the optimization process and the calculation of the loss function, and explains the theoretical foundations of the evaluated regularization techniques.

Chapter 3 presents the state of the art and the review of scientific literature, analyzing how regularization has been historically evaluated and justifying the purpose of this study.

Chapter 4 describes the experimental methodology: network architecture, datasets, internal activation metrics and the five designed experiments.

Chapter 5 presents the results of the five experiments with direct parallelism to the methodology: external performance (*accuracy* on CIFAR-10, SVHN, CIFAR-100 and FashionMNIST), internal activation metrics, temporal evolution, and robustness to noise in data and in weights. At the end of the chapter, the findings are integrated into a cross-sectional discussion that articulates the relationships between evaluation dimensions and places them in the context of the literature.

Chapter 6 gathers the conclusions of the work, answering the three formulated research questions, and proposes the main lines of future work.

THEORETICAL FRAMEWORK

2.1 Neural networks for deep learning

This chapter establishes the theoretical foundations necessary to understand the functioning of neural networks and the regularization techniques analyzed in this study. It explores everything from the basic concepts of deep learning to the internal metrics proposed to monitor the evolution of the model.

2.1.1 Concept

As mentioned previously, one of the most widely used machine learning algorithms today is neural networks, which are computational models inspired by the human brain. More technically, a dense neural network (or multilayer perceptron) is composed of a series of neurons organized in layers that apply, according to some weights (**weights**, which determine the strength of the «synaptic» connections between neurons) and biases (**biases**, added factors), linear transformations to the outputs of the previous layer (or to the input data in the first layer), followed by strictly non-linear functions called «activation functions».

Given an input vector $\mathbf{x}$, the activation $\mathbf{h}^{(l)}$ in the hidden layer l is defined as:

$$\mathbf{h}^{(l)} = \sigma(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}) \quad (2.1)$$

where $\mathbf{W}^{(l)}$ represents the weight matrix, $\mathbf{b}^{(l)}$ the bias vector, and σ is a non-linear activation function, such as the ReLU (*Rectified Linear Unit*), defined as $\sigma(x) = \max(0, x)$.

The chaining of multiple hidden or internal layers allows the network to project the original data into a high-dimensional latent space where complex patterns become linearly separable.

2.1.2 Training process

The training process of a neural network consists of finding the optimal set of parameters (weights and biases) that minimize the discrepancy between the model's predictions and the actual data labels, which is quantified by a loss (or cost) function that seeks to be minimized through an optimization process.

$$\theta^* = \arg \min_{\theta} (loss(D_{train}, \theta)) \quad (2.2)$$

where θ represents the network parameters and D_{train} is the training dataset. For classification problems, which are the only ones considered in this work, the most common approach in the literature is to use the Cross-Entropy Loss function. If y_i is the actual probability distribution (typically a *one-hot* vector) and $\hat{y}_i$ is the probability predicted by the network through a Softmax function (a function widely used in the output layers of neural networks because it allows converting output values to interpretable

probabilities), the loss is defined as:

$$L_{CE} = - \sum_i \mathbf{y}_i \log(\hat{\mathbf{y}}_i) \quad (2.3)$$

This is the function that is iteratively optimized during training, minimizing this L_{CE} function, usually through the Gradient Descent algorithm (or other slightly more modern or faster variants), passing the training data in batches (*batches*) and adjusting the weights and biases by passing the gradient of the loss function backwards through the network using the Backpropagation algorithm (*Backpropagation*).

2.1.3 Application to images: Convolutional neural networks

The neural networks described above process the input space through fully connected layers of neurons (*fully connected*), implying that each neuron receives information from all neurons in the previous layer. This scheme, despite being effective for «tabular» or low-dimensional data, is inefficient when the input is, for example, an image: a size of 32×32 pixels with three color channels already generates 3,072 input features, and the number of parameters of a single dense layer scales approximately quadratically with respect to the input dimension (in the case where the number of output neurons is of the same order as the input).

Furthermore, the use of fully connected networks on this type of data requires flattening the image into a one-dimensional vector, which breaks the spatial adjacency of the pixels and prevents exploiting the local structure of the image.

Convolutional Neural Networks (*Convolutional Neural Networks*, CNNs) address this problem by exploiting two intrinsic properties of visual data: the *locality* of spatial correlations and the *translation invariance* of the patterns to be detected.

Convolution and filters

The fundamental building block of a CNN is the convolutional layer. Instead of learning a weight matrix of dimension $n_{\text{input}} \times n_{\text{output}}$, each convolutional layer learns a set of K filters or kernels (*kernels*) of reduced size $F \times F \times C$, where F is the spatial dimension of the filter and C is the number of input channels. The discrete convolution operation of a filter $\mathbf{w}^{(k)}$ over an input $\mathbf{X}$ is defined, for each spatial position (i, j) , as:

$$z_{i,j}^{(k)} = \sum_{u=0}^{F-1} \sum_{v=0}^{F-1} \sum_{c=1}^C \mathbf{w}_{u,v,c}^{(k)} \cdot \mathbf{X}_{i+u,j+v,c} + \mathbf{b}^{(k)} \quad (2.4)$$

where $\mathbf{b}^{(k)}$ is the bias associated with filter k . The resulting activation $\mathbf{a}_{i,j}^{(k)} = \sigma(z_{i,j}^{(k)})$ is obtained by applying a non-linear activation function just as in conventional multilayer perceptrons.

This design incorporates two main advantages with profound implications for the learning capacity of the model. First, each neuron in the convolutional layer only processes a specific spatial region of the input, ignoring the rest. Second, **weight sharing**: the same filter $\mathbf{w}^{(k)}$ is applied to all positions (i, j) of the input map, so that the total number of trainable parameters of the layer is $K \cdot (F^2 C + 1)$, regardless of the image resolution. This reduction in parameter space, along with the ability to detect the same pattern at any position in the image, is one of the most powerful properties of CNNs.

Feature maps

The application of the K filters on the input data produces K two-dimensional matrices called *feature maps* (*feature maps*). Each map $\mathbf{A}^{(k)}$ encodes a specific pattern learned by the filter $\mathbf{w}^{(k)}$ (its presence and intensity) at each position (pixel) of the image. The first convolutional layers of a network tend to learn low-level detectors: edges, color gradients or corners, while the deeper layers, by processing the feature maps of previous layers, combine patterns to build representations of «higher abstraction», such as textures or complex shapes (the representational power of CNNs is based on this composition of convolutional layers in depth, with each layer operating on the *features* produced by the previous one to obtain a higher value representation of the data).

Pooling layers

To reduce the spatial dimension of the feature maps and, simultaneously, add a certain degree of local invariance to small translations, interleaved *pooling* layers are usually introduced. The most common variant is *max-pooling*, which divides the feature map into non-overlapping windows of size $P \times P$ and saves only the maximum value of each window:

$$\hat{\mathbf{A}}_{i,j}^{(k)} = \max_{\substack{0 \leq u < P \\ 0 \leq v < P}} \mathbf{A}_{iP+u, jP+v}^{(k)} \quad (2.5)$$

With $P = 2$, *max-pooling* halves the spatial resolution, decreasing the number of activations by a factor of four and producing more compact representations that are tolerant to small shifts.

2.2 The Problem of Overfitting and Generalization

Due to the large number of parameters that modern deep neural networks possess, their representation capacity usually far exceeds the complexity of the training data. This triggers the phenomenon of overfitting (*overfitting*), where the model minimizes L_{CE} by memorizing the noise or peculiarities of the training set, thus losing its ability to learn more general patterns and make quality predictions on new or unseen data.

2.3 Regularization Techniques

To mitigate overfitting, mechanisms are applied that restrict the model's capacity to memorize, through any modification made to the learning algorithm to reduce the generalization error [3], even if this implies not fully minimizing the training error. There are several ways to apply this regularization: we can directly alter the loss function formula (adding penalties), modify the network connections, alter the training flow or even alter the input data to force the model to learn more varied patterns. The main techniques used in this work are detailed below.

Although the following seven techniques (norm-based penalties, noise injection, Dropout, early stopping, data augmentation, and batch normalization) constitute the core of this study, it is important to note that the range of strategies is much broader and is constantly evolving. Likewise, there are more recent paradigms aimed at directly altering the training labels or objectives, such as *Label Smoothing*, or techniques based on the mixing of features and samples like *MixUp* or *CutMix* [4]. The specific techniques methodologically evaluated in this work are detailed below.

2.3.1 L_1 Penalty (Lasso)

L_1 regularization adds to the original loss function the sum of the absolute values of all network weights, scaled by a hyperparameter λ that controls the intensity of the penalty:

$$J(\theta) = L_{CE} + \lambda \sum_i |\mathbf{w}_i| \quad (2.6)$$

The effect of this penalty is that it forces sparsity (*sparsity*) in the weights, that is, when optimizing this function, many of the network weights are pushed towards zero, which, in practice, acts as an implicit feature selection mechanism, deactivating the least relevant connections for prediction.

2.3.2 L_2 Penalty (Ridge)

In the same way, L_2 regularization penalizes the magnitude of the parameters, but using the sum of the squares of the weights:

$$J(\theta) = L_{CE} + \lambda \sum_i \mathbf{w}_i^2 \quad (2.7)$$

Instead of forcing weights to zero, the derivative of this term penalizes weights with large values much more severely, forcing the network to distribute the learning more uniformly among all its connections. This results in a network with smoother activations that is less prone to react drastically to small variations in the input data. In neural networks, this method is commonly known as *Weight Decay*.

2.3.3 Dropout

Dropout is a structural regularization technique that dynamically modifies the topology of the network during training [5]. In each iteration (*batch*), every neuron in the hidden layers has a probability p of being «turned off» (ignored). Mathematically, this is equivalent to multiplying the activations by a mask drawn from a Bernoulli distribution:

$$\tilde{\mathbf{h}}^{(l)} = \mathbf{r}^{(l)} * \mathbf{h}^{(l)}, \quad \mathbf{r}^{(l)} \sim \text{Bernoulli}(p) \quad (2.8)$$

where $*$ represents an element-wise multiplication. This forces each neuron to extract robust features independently, since it cannot rely on the presence of specific other neurons for prediction. During the inference phase (test), neurons are not turned off, but rather the weights are scaled proportionally to p .

For convolutional neural networks, the direct application of standard *Dropout* to feature maps presents a major limitation: given that activations of spatially adjacent positions in a feature map are strongly correlated, the random deactivation of individual units removes local information but does not necessarily interrupt the signal of correlated features flowing through neighboring positions. To correct this deficiency, Tompson et al. ([6]) proposed *Spatial Dropout* (also called *Dropout 2D*), a variant in which the Bernoulli mask is applied at the level of the entire feature map rather than at the level of the individual activation.

Thus, with probability p the entire feature map corresponding to filter k is set to zero, entirely removing the contribution of that pattern detector for the current example. This coarser granularity is much more effective in CNNs, since it forces the network to learn redundant representations at the channel level, rather than at the pixel level, preserving the spatial coherence of convolutional learning.

2.3.4 Early Stopping

Early stopping is a form of implicit regularization. As a deep neural network trains, its training error continuously decreases; however, at a critical point, the validation error begins to increase, leading to overfitting. *Early Stopping* consists of monitoring this validation metric during training epochs and halting the optimization algorithm when performance stops improving after a predefined number of epochs («patience»), which restricts the level of memorization the model can achieve and prevents the weights from overly adapting to the noise of the training set.

2.3.5 Gaussian Noise Injection (*Gaussian Noise*)

Noise injection is an effective strategy for improving the robustness of internal representations. It consists of adding noise to the model's inputs. Formally, given an input x , the network processes a corrupted version $\tilde{x}$:

$$\tilde{x} = x + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma) \quad (2.9)$$

By training with slightly altered samples in each iteration, the algorithm tends to learn the latent shape of the data somewhat better instead of memorizing exact points, obtaining a simpler decision boundary.

Occasionally, noise can be applied directly to the network's hidden weights, (this will be an experiment conducted in this study), consequently modifying the feature maps learned by each layer. Formally, the weights w_i are replaced, at evaluation or training time, by perturbed versions:

$$\tilde{w}_i = w_i + \eta_i, \quad \eta_i \sim \mathcal{N}(0, \sigma_w) \quad (2.10)$$

The theoretical justification for this technique connects with the *flat minima* framework [7, 8]: models whose loss function minima are broad and flat in the parameter space tend to generalize better than those whose minima are narrow and sharp, since a small perturbation of the parameters does not significantly degrade their performance. Thus, the optimization process is guided toward more robust parameter solutions. In this work, weight perturbation will be used as an experiment to examine the relationship between this robustness and generalization error.

2.3.6 Data Augmentation

This technique is based on the principle that having more training data (of quality, that is, more varied) is the best way to reduce overfitting. *Data Augmentation* aims for this by creating synthetic yet realistic variations of the existing data. In the image domain, this is achieved by applying transformations such as rotations, translations, random crops (*crops*), horizontal flips, or alterations in contrast and brightness, among many others. Faced with so many variations, the neural network is forced to develop more invariances, learning robust semantic patterns regardless of the position, scale, or other more «irrelevant» factors of the object in the image. For classical tabular data, other transformations can be scale changes or noise injections similar to the previous case.

2.3.7 Batch Normalization

Batch Normalization (*Batch Normalization*) was originally proposed to accelerate the training of deep networks by mitigating *Internal Covariate Shift* [9], that is, the variation in the distribution of the activa-

tions of the hidden layers during learning. This technique spatially normalizes the activations produced by a layer based on the statistics of the current batch (*mini-batch*).

Given an activation x produced in a layer, *Batch Normalization* applies the following linear transformation:

$$\begin{aligned}\hat{\mathbf{x}} &= \frac{\mathbf{x} - \boldsymbol{\mu}_B}{\sqrt{\boldsymbol{\sigma}_B^2 + \epsilon}} \\ \mathbf{y} &= \gamma \hat{\mathbf{x}} + \boldsymbol{\beta}\end{aligned}\tag{2.11}$$

where μ_B and σ_B^2 are the mean and variance calculated empirically over the current batch, ϵ is a numerical stability constant, and γ and β are learnable network parameters that allow scaling and shifting the normalized distribution to recover the original representational capacity if the model requires it.

Although its primary motivation was not regularization, in practice it acts as a powerful implicit regularizer. Since the *mini-batch*-dependent mean and variance act as noisy estimates of the global statistics of the dataset, *Batch Normalization* injects stochastic multiplicative and additive noise at each training step. This noise reduces the deterministic dependence of the network on specific training examples, improving generalization and often reducing or eliminating the need to use techniques like *Dropout*.

2.4 Measures for the dynamics of internal activations

To understand how regularization techniques truly affect the learning process, it is imperative to analyze the intrinsic behavior of the network. If we consider a hidden layer l , its activation vector $\mathbf{h}^{(l)}$ for a dataset can be mathematically treated as a set of continuous random variables. The study of how the statistical properties of these variables evolve over training epochs and under different techniques constitutes the dynamics of internal activations.

2.4.1 Probability Distributions and Entropy

Since the activation space is continuous and high-dimensional, the analysis is performed **at the full-layer level**. The first step consists of estimating its probability density function by quantifying the value space into discrete intervals (*bins*), modeling the activations as a discrete probability distribution. In convolutional layers, this analysis is applied to the spatial average of the feature maps.

Once modeled, we apply concepts from Information Theory to evaluate their representational capacity. The fundamental metric is the Shannon Entropy, which for a discrete distribution $\mathbf{P}$ with N intervals is defined as:

$$H(\mathbf{P}) = - \sum_{i=1}^N p_i \log_2(p_i)\tag{2.12}$$

where p_i is the empirical probability that an activation falls into interval i [10]. Entropy quantifies the richness of the representation: a high entropy indicates that the activations are uniformly distributed, taking advantage of the layer's capacity. Conversely, a low entropy reveals a concentrated distribution,

typical of regularizers like L_1 where the probability collapses to zero. In addition, this process allows calculating the **percentage of unique states**, a metric that we will present in the design chapter to identify how many different activation signatures or patterns the model generates.

2.4.2 Activation Dispersion

Along with entropy, the model's stability is linked to the **global dispersion of its activations**. We quantify the magnitude of the signals using the empirical standard deviation (σ) of all activation values in a layer l :

$$\sigma^{(l)} = \sqrt{\frac{1}{M} \sum_{j=1}^M (\mathbf{h}_j^{(l)} - \mu^{(l)})^2} \quad (2.13)$$

where $\mu^{(l)}$ is the layer's mean activation and M is the total number of evaluated activations (which is the number of neurons times the number of validation images passed). In this work, the **dispersion ratio** $\sigma^{(l)} / \sigma_{\text{baseline}}^{(l)}$ is used, which normalizes the deviation of each method with respect to the model without regularization. This ratio facilitates the interpretation of compression: for example, if the *baseline* model has a $\sigma = 10$ and a regularized model has $\sigma = 4$, the 0.4 ratio indicates that the technique has reduced the dispersion to less than half. Uncontrolled dispersion is a symptom of instability, a phenomenon known as *Internal Covariate Shift* [9].

STATE OF THE ART

3.1 Generalization in deep models

Understanding why deep neural networks generalize well is, surprisingly, an open problem in the field. Classical statistical theory links the generalization capacity of a model to its relative size compared to the dataset: models with more parameters than examples should overfit severely. In practice, deep networks systematically violate this prediction, generalizing correctly despite having several orders of magnitude more parameters than training samples [3].

The work of Zhang et al. (2017) [1] turned this intuition into undeniable experimental evidence. The authors trained reference architectures on CIFAR-10, but progressively replacing the real labels with randomly assigned labels or, in the most extreme experiments, substituting the images with pure random noise. In all cases, the networks achieved zero training error: they are capable of memorizing any arbitrary input-output mapping, regardless of whether there is any real statistical structure in the data. And yet, those same architectures, trained with real data, generalize with high accuracy.

This has a direct consequence for the study of regularization. If classical capacity frameworks, such as VC dimension [11] or Rademacher complexity [12], do not explain generalization, they also cannot accurately explain why regularization improves it. It is not enough to say that regularization reduces the model's complexity: what it does is induce inductive biases that guide the optimizer towards solutions that, among all those that fit the data, also generalize.

The relevant question then becomes what kind of solutions each technique favors and how that bias manifests in the internal structure of the model.

3.2 Empirical evaluation of regularization

Recent literature has produced *surveys* that organize the growing catalog of available techniques. Santos and Papa (2022) [13] propose a taxonomy oriented towards convolutional networks that distinguishes between methods acting on the loss function, methods modifying the topology or information flow of the network, and methods intervening on the input data itself. Moradi et al. (2020) [14] apply this framework to deep models in general, incorporating normalization and network weight initialization techniques as implicit forms of regularization. This opens the door to extending such taxonomies by incorporating criteria based on the internal dynamics of representations, like the ones used in this work.

Within explicit penalties, L_1 and L_2 regularization accumulate a long history of comparative studies. Deepa et al. (2025) [15] evaluated both techniques, alongside their combination (*Elastic Net*), in image classification tasks and concluded that no variant universally dominates the others: the relative efficacy depends on the architecture, the dataset, and the hyperparameters used. Along the same lines, Mohamed et al. (2025) [16] propose a smoothed variant of the L_1 penalty for higher-order networks, demonstrating that it stabilizes the optimization process measured in terms of training error

convergence.

Dropout, whose original formulation is due to Srivastava et al. (2014), [5] constitutes the reference for regularization by «changing» the network structure. Although the original work includes activation histograms showing how the random deactivation of neurons distributes the internal representations more uniformly, this analysis is presented qualitatively and does not systematically correlate with generalization performance, leaving open the question of what ranges of internal activation are diagnostic of effective regularization. This type of analysis could be expanded by incorporating quantitative metrics of internal activation, like those employed in this work.

The authors demonstrated its effectiveness in computer vision, language modeling, and audio recognition, always by comparing loss and accuracy curves in training and testing. Subsequent comparative studies, such as the one by Mejía Cajicá et al. (2020) [17], replicated and extended these results including *Batch Normalization* and *Early Stopping*, with the same methodological apparatus: training on standard datasets and comparing external performance metrics on the validation or test set.

This methodological convergence is a relevant fact. Practically all empirical studies on regularization—including those cited above, as well as Ismoilov and Jang (2018) [2] in their comparative analysis on meteorological time series— evaluate a technique’s success through metrics calculated at the end of training, from outside the model. This approach, which we will refer to here as *black-box* evaluation, responds to an evident practical need: in production, what matters is the quality of the predictions. However, it offers little information about the mechanism by which regularization acts. It does not allow, for example, distinguishing whether two techniques that produce the same final accuracy do so by inducing similar or radically different internal representations.

Modularization, reviewed by Amer and Maul (2019) [18] as an alternative complexity control strategy, points in a different direction: instead of penalizing or perturbing, it reorganizes the model structure. That this approach is conceptually distinct but evaluated with the same external criteria illustrates the extent to which the evaluation paradigm is cross-cutting throughout the literature, regardless of the type of regularization studied.

3.3 Domain dependence and performance variability

One of the most consistent conclusions of empirical studies is that there is no universally superior regularization technique. The analysis by Ismoilov and Jang (2018) [2] on meteorological data showed that the advantages *Dropout* and batch normalization typically exhibit in computer vision reduce or disappear in domains with very different statistics, such as time series. Similar results are observed in the work of Mejía Cajicá et al. (2020) [17] where the ranking of techniques varies according to the dataset and architecture depth.

This variability, from the perspective of Zhang et al., is normal: [1] if regularization induces inductive biases, its efficacy depends on whether those biases are consistent with the statistical structure of the problem. What is significant is that empirical literature has not yet found a systematic way to predict a priori which technique will work better in a given context, beyond experimental testing. The *black-box* evaluation allows identifying what works, but not *why* it works in some domains and not in others.

3.4 Towards a «total» evaluation

There is a smaller line of research that directly addresses the internal properties of representations during training.

The most influential starting point is the *Batch Normalization* of Ioffe and Szegedy (2015). [9] Their proposal introduced an effective technique and a hypothesis about it: the improvement in training would be due to the reduction of *Internal Covariate Shift* (ICS), that is, the change in the distribution of activations of each layer caused by the update of previous parameters. It was the first time a high-impact regularization work argued its effectiveness in terms of what happens inside the model.

Santurkar et al. (2018) [19] questioned that hypothesis experimentally. By introducing deliberate noise that destroyed the stabilization of the activation distribution, nullifying the anti-ICS effect, the model's performance was barely affected. Their conclusion was that the real benefit of batch normalization lies in smoothing the loss landscape, which makes the gradient more stable.

This proves that hypotheses about the internal workings of regularization can and must be empirically verified, and shows that the distribution of internal activations can change in ways not immediately reflected in external metrics.

More recently, Brown et al. (2022) [20] demonstrated that common regularization techniques consistently reduce the *Local Intrinsic Dimensionality* (LID) of the last layer's activations, and that this reduction strongly correlates with the model's generalization capacity.

They also found that during the *grokking* phenomenon (the sudden improvement in validation accuracy after many epochs of apparent stagnation) a simultaneous drop occurs in the intrinsic dimension of activations, an event that external metrics do not anticipate. Liu et al. (2023) [21] proposed the *Diffusion Spectral Entropy* (DSE) as an entropy measure adapted to the geometry of high-dimensional representations, observing that this entropy systematically increases during training in models that generalize correctly, while it remains stagnant in models that overfit; the spectral mutual information derived from this measure predicted classification accuracy in more than 900 models on ImageNet without access to the test set.

Finally, the work presented at ICLR 2024 under the intrinsic dimensionality regularization (IDRR) framework [22] explicitly incorporated the geometry of representations into the loss function, achieving 68–81% reductions in the effective dimensionality of representations without significant loss of accuracy.

Black-box evaluation is the dominant paradigm in the empirical regularization literature; the intrinsic analysis of representations exists as a very promising line but still lacks a large number of studies. In the following chapter, the experimental methodology followed in this work will be presented, proposing new viewpoints and measures to link the geometry of internal activations to the regularization effect. This study aligns with recent proposals like DSE [21], using entropy measures to diagnose the quality of learned representations.

EXPERIMENTAL DESIGN AND METHODOLOGY

The architecture and experimental design developed to answer the questions raised in this work are detailed below.

The objective of the design is to create a reproducible and modular framework of experiments in which the learning, extraction, and activation analysis phases can be decoupled.

4.1 Neural network architecture

To force a positive effect of regularization, the selected network must meet the requirement of being overparameterized with respect to the complexity of the data. If the model lacks the necessary capacity to memorize the training set (and, therefore, overfit), the effect of the regularization techniques would be «covered» by the network's own error (*underfitting*).

Therefore, a Convolutional Neural Network (CNN) has been designed, composed of a depth of three convolutional blocks followed by a dense classification stage. Throughout all experiments, this topology is kept unchanged so as not to alter comparisons and obtain consistent results. The main blocks are divided as follows:

- **Feature extraction:** Composed of a sequence of three convolutional layers. Each layer progressively increases the depth of the channels (32, 64, and 128 filters respectively) to capture patterns of greater abstraction. 3×3 *kernels* are used with *padding* adjusted to preserve spatial dimensionality before subsampling. After each pass through the weights, the ReLU activation function is applied, followed by 2×2 *Max-Pooling* operations to obtain invariance to small local translations.
- **Classification layer:** The resulting tensors from the last convolutional block are flattened (*flatten*) and projected onto a multilayer perceptron. This stage consists of a dense layer of 128 neurons (f_{c1}) that acts as the definitive latent space before the output layer (f_{c2}), where the inter-class probability distribution is calculated using a *softmax* function.

Table 4.1 summarizes the specific architecture employed. The choice of three convolutional blocks with 32, 64, and 128 filters respectively produces a network with enough capacity to overfit CIFAR-10 (verified empirically: the *baseline* reaches *train_acc*= 1.0 at epoch 60), while its moderate size allows for agile and reproducible training.

4.2 Evaluation with multiple datasets

Since the objective of regularization is to add inductive biases that favor generalization, the literature has amply demonstrated that the efficacy of a technique is not universal [2, 17], but presents a strong dependence on the dataset (*dataset*). Evaluating regularization techniques on a single empirical prob-

Layer	Type	Parameters	Output
conv1	Conv2d + ReLU + MaxPool	3→32, kernel 3×3	16×16×32
conv2	Conv2d + ReLU + MaxPool	32→64, kernel 3×3	8×8×64
conv3	Conv2d + ReLU + MaxPool	64→128, kernel 3×3	4×4×128
Flatten	—	—	2048
fc1	Linear + ReLU	2048→128	128
fc2	Linear (+ Softmax)	128→10	10

Table 4.1: Architecture of the CNN used in all experiments. The output sizes correspond to $32 \times 32 \times 3$ inputs (CIFAR-10 and SVHN).

lem would lead to biased conclusions, so the code has been developed to be able to replicate the experiments on different sets of images. In particular, during this study the following have been used:

- 1.— **CIFAR-10** [23] Set of 60 000 RGB images (50 000 training and 10 000 testing) of 32×32 pixels, divided into 10 classes (like airplane, car, etc.).
- 2.— **CIFAR-100** [23] Set of 60 000 RGB images (50 000 training and 10 000 testing) of 32×32 pixels, organized into 100 classes. Each class also belongs to one of 20 superclasses.
- 3.— **SVHN** (*Street View House Numbers*) [24] Set of 32×32 pixel RGB images that contain digits from 0 to 9 extracted from real-world images (numbers on house facades). The complete dataset reaches $\sim 630\,000$ images.
- 4.— **FashionMNIST** [25] Set of 70 000 grayscale images (60 000 training and 10 000 testing) of 28×28 pixels, distributed in 10 classes of clothing (like t-shirts, trousers, shoes, etc.).

All datasets are normalized before use. This process consists of scaling the pixel values (originally in the $[0, 255]$ range) to the $[0, 1]$ interval by dividing each value by 255. A restriction of 25 000 images has been established for each training set, for performance and execution time reasons, although the validation sets remain complete, with 10 000 images each, except for the SVHN case, where it is limited to the same number since its original validation set has 26 032 samples.

4.3 Hyperparameter search

To evaluate the regularization techniques by their strength, a grid search (*Grid Search*) has been carried out on their hyperparameters. This allows analyzing how the model's behavior changes as the regularization intensity increases.

The evaluated ranges have been defined following common values in practice:

- **L_1 and L_2 regularization:** λ is modified in logarithmic scale, in the range $\lambda \in [10^{-4}, 0.2]$.
- **Dropout:** Dropout probabilities p in the interval $[0.1, 0.6]$ are used in both dense and convolutional layers.
- **Gaussian noise:** Noise with standard deviation $\sigma \in \{0.01, 0.05, 0.1, 0.2\}$ is added during training.
- **Early stopping:** Training is stopped after 3, 5, 8, 12 and 15 epochs without improvement in validation.
- **Data augmentation:** For this study, data augmentation values have been discretized into 0 (off), 1 (*random crop*), 2 (*horizontal flip*) and 3 (*random crop and horizontal flip*). The param-

eters of each transformation have been kept fixed (0.5 probability for *RandomHorizontalFlip* and random crop of size 32 with padding of 4 in *RandomCrop*, without performing hyperparameter tuning to analyze exclusively the aggregate effect of the augmentation strategies.

- **Batch normalization:** For this method, no parameter is required, so only turning it on or off is tested.

4.4 Training pipeline

To ensure consistent comparisons, all models are trained following the same procedure. The loss function used is cross-entropy (*Cross-Entropy Loss*). The optimizer employed is Adam (*Adaptive Moment Estimation*, [26]), which automatically adjusts the learning rate of each parameter based on gradient estimates, allowing for fairly rapid convergence and making it widely used in the scientific community.

The batch size (*batch size*) is fixed at 64, meaning that the network weights are adjusted once every 64 data steps, and the network is trained for 60 epochs to attempt to force overfitting on simple problems, without using *early stopping* except when a specific experiment requires it, to observe the model's behavior throughout the entire process and precisely avoid not overfitting.

During training, which is done through the *PyTorch* module, the model weights are saved at each epoch (*checkpoints*) in `.pth` format, so that later the internal behavior of the network can be analyzed without interfering in the training process. In addition to being able to load the models in evaluation mode later to study internal activations (using *PyTorch hooks*, see next section), we can also modify the weights (for example, adding noise) to analyze the stability of the models.

This complete procedure allows us to perform fair and reproducible comparative experiments (as long as we fix a common random seed, to guarantee consistency in processes such as weight initialization or data passing).

4.5 Activation extraction system

Once the models are saved (*checkpoints*), the next step consists of extracting the activations from the hidden layers. This process must be carried out without modifying the model's behavior or introducing overhead during training.

To do this, a system based on *forward hooks* is used, which allows capturing the outputs of specific layers during the forward pass.

The procedure is as follows:

- 1.– **Evaluation mode:** The model is instantiated and the weights corresponding to a specific point in training are loaded. Next, evaluation mode (*eval*) is activated, which deactivates components such as *Dropout* and fixes the behavior of *Batch Normalization*, obtaining deterministic outputs.
- 2.– **Layer registration:** *Hooks* are added to the layers of interest; in this case, the first and last convolutional layers («conv1» and «conv3») and the main dense layer before the output («fc1») have been selected.
- 3.– **Activation extraction:** During inference on the validation set, the *hooks* capture the outputs of the selected layers. These activations are copied outside the network and stored for subsequent analysis.

This approach allows analyzing the internal representations of the model at different points in training without affecting it.

4.6 Activation quantification: metrics

Based on the extracted activations, an analysis of the internal representations of the model is performed using different metrics and experiments to characterize how information evolves throughout the network and how it is affected by different regularization techniques and parameters.

4.6.1 Discretization and vectorization of internal states

In the early stages of the design, an analysis based on global frequency histograms was proposed, which, similarly to a «bag of activations» (BoW model), counted how many times each activation (a small range of them, *bin*) appeared. However, this method leads to a loss of information, since two «opposite» states (some neurons activate and others deactivate and vice versa) could result in the same histogram. Therefore, a specific discretization process has been designed to compare internal states by constructing an ordered vector. In this vector, each component represents a specific neuron, and its integer value indicates the identifier of the *bin* into which its activation falls.

The limits of the *bins* are not recalculated for each model, but are obtained beforehand from the minimum and maximum activations observed in the unregularized *baseline* model on the validation set. Once this global interval is determined, it is uniformly divided into N *bins*, which remain fixed throughout the experimental comparison.

As an illustrative example, if a hidden layer composed of only 3 neurons were evaluated and we grouped their activations in a binary way (activated/deactivated in 2 *bins*), a representative vector of the internal state could be (1, 0, 0). This result would indicate that the activation of the first neuron given an image has fallen into the second *bin* (value 1), while the activations of the other two have remained in the first (value 0).

On this vectorial format of discretized states, similarity and diversity calculations between states will subsequently be executed.

From the collection of state vectors obtained for all examples of the evaluation set, the percentage of unique states (*unique_pctg*) is calculated: the proportion of different vectors out of the total processed examples. A layer that generates very diverse representations will present *unique_pctg* values close to 1 (almost every example produces a different internal state), while a layer with highly compressed or collapsed representations will show low values (many examples share the same discretized state). This metric will be used in the time evolution experiment to study how representational diversity varies throughout training.

4.6.2 Shannon entropy and activation dispersion

For each layer, all activations generated for all images in the validation set are jointly collected and the histogram is found from which its global entropy and standard deviation are calculated.

From the activation distributions, the Shannon entropy of each layer is calculated. For this, a frequency histogram is built over the previously defined N *bins* and normalized to obtain a discrete probability distribution p_i . The entropy reported for each layer, method, and regularization value corresponds

to the entropy of the computed global histogram, using the calculation detailed in formula 2.12, and considering only the *bins* with non-zero probability.

Thus, it can be observed whether the activation is concentrated in a small number of hidden states or if, conversely, it is distributed more evenly. In the first case, the entropy is low and the representation is dominated by few features; in the second, the entropy is higher and the information is spread among more components.

In addition, the dispersion of the activations is analyzed through the empirical standard deviation (σ), to study how stable the network responses are and how they vary between different inputs. The hypothesis is that excessive dispersion can hinder learning and affect model stability, pointing to a case of *overfitting*, while smaller values may indicate more consistent behavior (without reaching the extreme of being too compressed values, in which case the network would not be learning enough to solve the problem satisfactorily, *underfitting*).

Subsequently, this magnitude is normalized with respect to the *baseline* model without regularization through the quotient $\sigma/\sigma_{\text{baseline}}$.

4.7 Main experiments

Based on the experimental framework described up to this point (fixed architecture, hyperparameter grid search, activation extraction via *hooks*), five study lines are articulated that address different dimensions of the same training.

The first examines the external performance of the model (EXP-1). For each of the four considered datasets and for each *grid search* configuration, an evaluation of the model's *accuracy* in training and validation is made, to study how performance varies as the regularization intensity increases in these *datasets* and methods, also observing the generalization gap ($\text{train_acc} - \text{val_acc}$) and, above all, obtaining a reference baseline to interpret the following experiments.

Subsequently, the three proposed internal diversity metrics (Shannon entropy, dispersion ratio, and percentage of unique states) are cross-referenced with the validation *accuracy* (EXP-2). The objective is to determine if there is any range of these metrics in which the best performing models are consistently concentrated, which would be equivalent to identifying an internal diagnostic signal of regularization quality.

The third line incorporates the temporal dimension (EXP-3). Instead of studying only the state of the model at the end of training, the evolution of the percentage of unique states (*unique_pctg*) in the three layers is evaluated throughout the 60 epochs, for each regularization technique. This allows determining if the divergence between methods occurs gradually or if there are critical phases of training in which internal representations change abruptly.

The last two lines study the robustness of the trained models. In the first, Gaussian noise of increasing intensity is added to the images of the validation set while keeping the weights unaltered (EXP-4), to measure how much performance each model retains given degraded inputs. The hypothesis is that a model less overfitted to its training data will present a smoother degradation.

In the second, the noise is introduced directly onto the weights of specific layers during prediction (validation set), instead of onto the data (EXP-5). A sharp drop in performance in the face of small perturbations in the parameters would indicate that the model has converged to sharp minima (*sharp minima*) in the loss space, while a smooth degradation would point to flat minima (*flat minima*), associ-

ated in the literature with greater generalization capacity.

4.8 Computational environment and reproducibility

The entire system implementation has been done in Python using PyTorch. Both this and the other modules and libraries on which the work has relied can be consulted in the references section ([27–31]).

The training was executed on a GPU (NVIDIA GeForce RTX 3050) using CUDA, which allowed accelerating the execution of the tests (and explorations of various configurations or experiments to see which make sense and which do not) on various methods, parameters, datasets, etc.

To guarantee the reproducibility and transparency of this research, the complete source code, including training scripts, activation extraction, and graph generation, is publicly available in the following GitHub repository: <https://github.com/admsit0/tfg>

RESULTS AND DISCUSSION

This chapter presents the results of the five experiments described in the previous chapter and discusses them as a whole. The results take CIFAR-10 as the main domain and are contrasted with SVHN, CIFAR-100, and FashionMNIST.

5.1 Accuracy Results: Hyperparameter Search (EXP-1)

For each technique, a set of models is trained using *grid search* over the regularization hyperparameter; for each method, the model with the highest *val_acc* at the end of the 60 training epochs is retained.

The results reveal that, although differentiated performance levels are maintained, the effectiveness of each technique depends on the evaluated domain (Figure 5.1). **Data Augmentation** and **Batch Normalization** achieve the most significant improvements over the baseline in all scenarios. In an intermediate level are **Dropout** and **L2**, with moderate benefits. The third group (**Gaussian Noise**, **Early Stopping**, and **L1**) barely exceeds or even worsens the base performance.

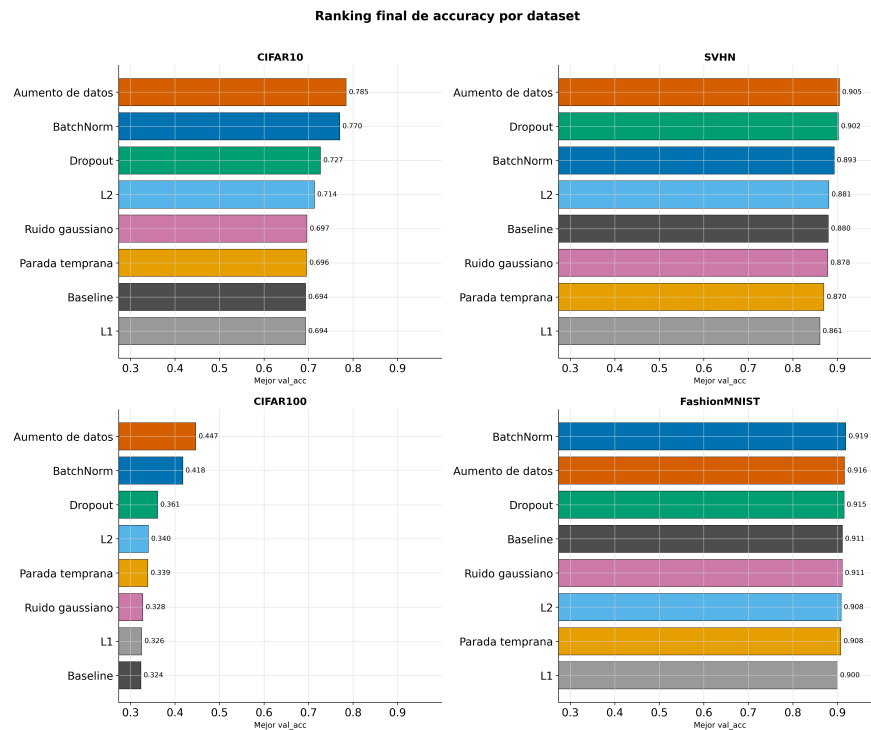


Figure 5.1: Final *val_acc* ranking for the four evaluated domains (CIFAR-10, SVHN, CIFAR-100, and FashionMNIST).

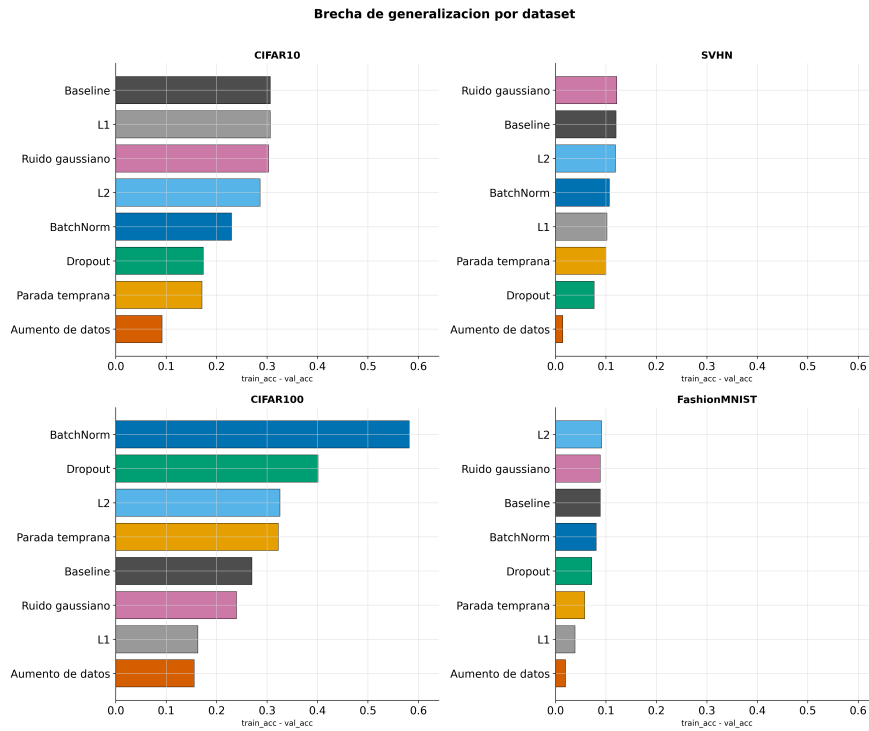


Figure 5.2: Generalization gap for the four evaluated domains (CIFAR-10, SVHN, CIFAR-100, and FashionMNIST).

Beyond the ranking, the **generalization gap** ($gap = train_acc - val_acc$), visible in Figure 5.2, clearly differentiates the methods. In CIFAR-10, the Baseline, L2, and Gaussian Noise achieve $train_acc \approx 1.0$ with a gap close to 0.30, indicating memorization of the training set. DataAug reduces the gap to ~ 0.09 evidencing implicit regularization. BatchNorm maintains a gap of ~ 0.23 , suggesting that its advantage comes from stabilizing the optimization. Early Stopping halts training at epoch 11 (patience=8), pointing to the fact that overfitting is concentrated in late epochs.

The comparison in Figure 5.1 allows these observations to be extended. In SVHN, the absolute performance increases due to the greater homogeneity of the images, but the relative ranking is preserved. In CIFAR-100, the complexity of the problem causes performances to drop; DataAug (0.44) and BatchNorm (0.42) still lead, but the bottleneck is the model's capacity.

In FashionMNIST, the opposite effect is observed: all methods exceed 0.91 in val_acc with minimal differences (< 0.01), constituting a clear performance ceiling.

5.2 Internal Activations Analysis

The *accuracy* results measure the external effect of regularization. Following, the internal perspective is adopted: how each technique alters the distribution and diversity of the network's hidden representations.

5.2.1 Correlation between Internal Metrics and Performance (EXP-2)

The activation analysis characterizes the internal structure of the representations through three metrics: the **percentage of unique states** (*unique_pctg*), the **Shannon entropy** (H), and the **dispersion ra-**

tio. Although the experiments are carried out in three layers considered “representative” in the network, the results are more visual in the `fc1` layer, where the effect of regularization is shown more clearly in the considered metrics, compared to the stability of the earlier convolutional layers.

Visually, the effectiveness of regularization is manifested in the “compression” of the activation histograms. A narrower and peakier histogram indicates a more compact and regularized representation, avoiding the dispersion associated with the *baseline* in deep layers. This can be observed in Figure 5.3, where we can see how, in the `fc1` layer (bottom row), the activations are compressed from the range $[0, \sim 52]$ to the range $[0, \sim 13]$ in the regularized scenario. Both histograms present a lower limit at 0 as a direct consequence of the ReLU activation. More examples of histograms showing this phenomenon can be found in the Appendices of this work (A.1).

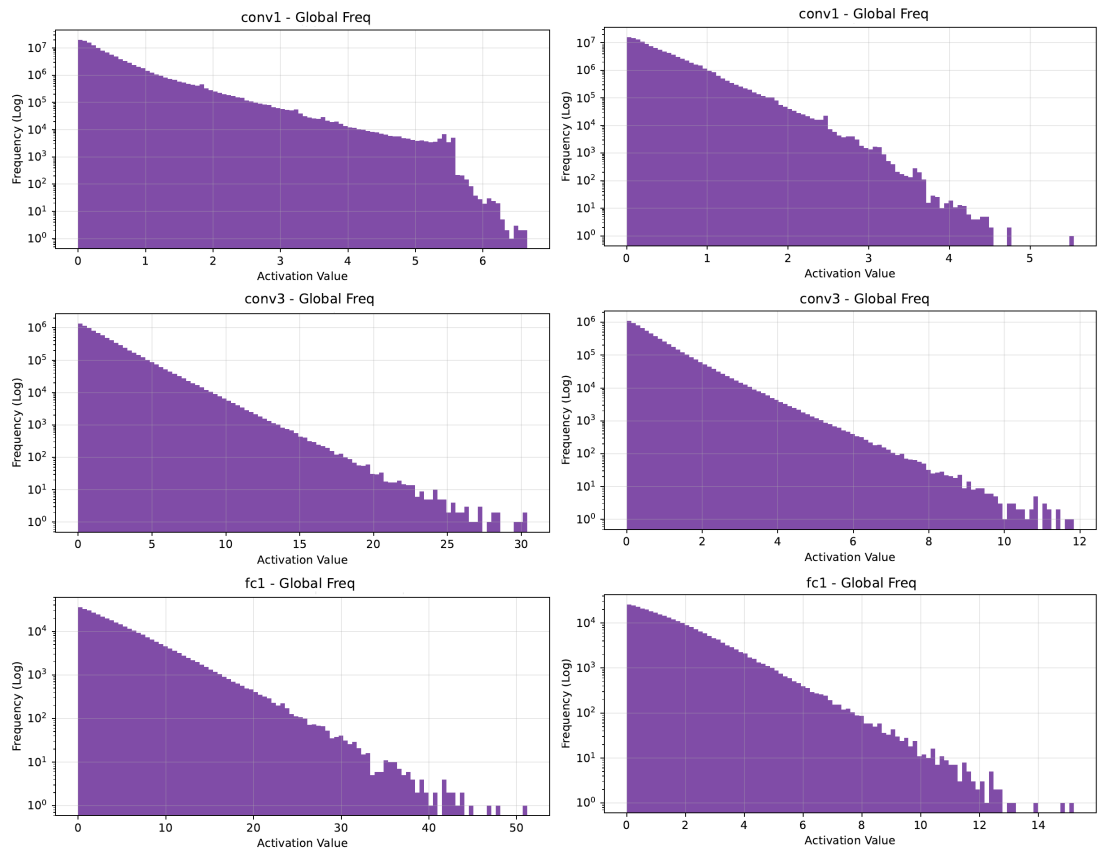


Figure 5.3: Distribution of activations by layer (`conv1`, `conv3`, and `fc1`), on the validation set of *CIFAR-10*. On the left, the histograms of the unregularized network, and on the right, the specific case with *Dropout* $p = 0.4$. It is observed how regularization tends to “compress” the distribution of activations, reducing the diversity of states to favor more generic representations.

While these visualizations allow qualitatively identifying the effect of regularization on activations, the systematic comparison between methods requires quantitative metrics that characterize the internal structure of the representations, and those mentioned above (*unique_pctg*, entropy, and dispersion ratio) will be used for this purpose.

In Table 5.1, these values are compared for the optimal scenario of each method in the four evaluated datasets. It is observed that the methods with the best performance (DataAug, BatchNorm, and Dropout) present intermediate diversity values and a dispersion ratio lower than one, suggesting a more compact and regularized representation. In contrast, the worst-performing methods (GaussNoise, Baseline) show high values in all metrics, indicating a chaotic dispersion of activations, while collapse

(EarlyStopping) drastically reduces diversity and entropy, which is reflected in a very low dispersion ratio.

For the calculation of entropy and unique states, 30 bins were used for CIFAR-10 and 10 bins for the rest of the datasets, since it was observed that, when using only 10 bins on the first dataset, the unique states tended to equalize too much, having too low volatility in the percentage of unique states and entropy to study patterns, which justifies a higher number of bins to adequately capture the distribution of activations and to be able to observe more clearly the changes in the internal measurements depending on the regularizing strength. It is worth noting that, although the number of bins affects the absolute values of the metrics, their relative ranking among methods tends to remain almost completely consistent, validating their use for the comparative analysis.

In the following figure (Figure 5.4), it can be seen that the pattern is consistent in the main datasets, showing that the best *accuracies* tend to be achieved for a dispersion in an intermediate range, and heavily dependent on the regularization method (see Dropout, with green marker, which along the test grid covers almost the entire intermediate zone in CIFAR-10 and SVHN). This pattern breaks in FashionMNIST, where most methods achieve very high precision with highly disparate internal metrics, and in CIFAR-100, where the model's representational capacity is not sufficient to learn the dataset well.

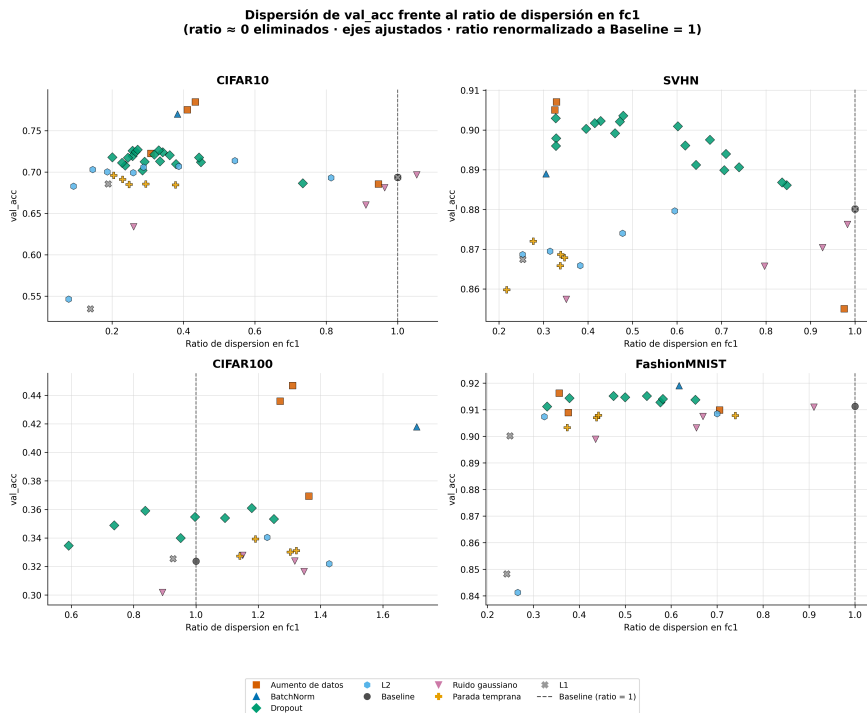


Figure 5.4: Dispersion of val_acc versus dispersion of f_{c1} for the four domains. The intermediate diversity zone concentrates the best results in CIFAR-10 and SVHN.

Method	val_acc	uniq_pctg (%)	Entropy H	Disp. ratio	bins
CIFAR-10					
Data Augmentation	0.785	89.22	1.78	0.46	30
BatchNorm	0.770	84.30	1.53	0.41	30
Dropout	0.727	37.23	1.17	0.29	30
L2	0.714	97.93	2.09	0.57	30
Gaussian Noise	0.697	100.00	2.93	1.11	30
Early Stopping	0.696	20.56	0.88	0.22	30
Baseline	0.694	99.98	2.87	1.06	30
L1	0.694	99.98	2.87	1.06	30
SVHN					
Data Augmentation	0.907	0.69	0.06	0.33	10
Dropout	0.904	2.74	0.19	0.48	10
BatchNorm	0.889	0.70	0.10	0.31	10
Baseline	0.880	19.75	0.79	1.00	10
L1	0.880	19.75	0.79	1.00	10
L2	0.880	5.63	0.33	0.59	10
Gaussian Noise	0.876	19.44	0.77	0.98	10
Early Stopping	0.872	0.68	0.03	0.28	10
CIFAR-100					
Data Augmentation	0.447	40.95	1.33	1.31	10
BatchNorm	0.418	75.10	1.61	1.71	10
Dropout	0.361	29.49	1.20	1.18	10
L2	0.340	39.45	1.30	1.23	10
Early Stopping	0.339	39.85	1.28	1.19	10
Gaussian Noise	0.328	34.42	1.23	1.15	10
L1	0.326	23.48	0.92	0.93	10
Baseline	0.324	27.16	1.17	1.00	10
FashionMNIST					
BatchNorm	0.919	15.34	1.08	0.62	10
Data Augmentation	0.916	1.76	0.74	0.36	10
Dropout	0.915	14.42	0.88	0.55	10
Baseline	0.911	82.72	1.44	1.00	10
Gaussian Noise	0.911	76.88	1.36	0.91	10
L2	0.908	37.50	1.14	0.70	10
Early Stopping	0.908	6.17	0.84	0.44	10
L1	0.900	1.64	0.62	0.25	10

Table 5.1: Activation metrics in $\mathbb{R}^1$ for the optimal scenario of each method in the four evaluated datasets.

5.2.2 Temporal Evolution of Representations (EXP-3)

It can also be interesting to study the temporal evolution of the internal metrics that we consider, since it can allow us to identify very important phases or patterns in the training of a network (when most of the learning is done, when overfitting occurs, etc.). As before, the interpretation will focus on the `fc1` layer, because the analyzed convolutional layers (`conv1` and `conv3`) show a very rapid evolution towards “saturation” (high entropy, dispersion, and percentage of unique states close to 100%).

Figure 5.5 details the evolution of *unique_pctg* during the 60 training epochs in the `fc1` layer of the network.

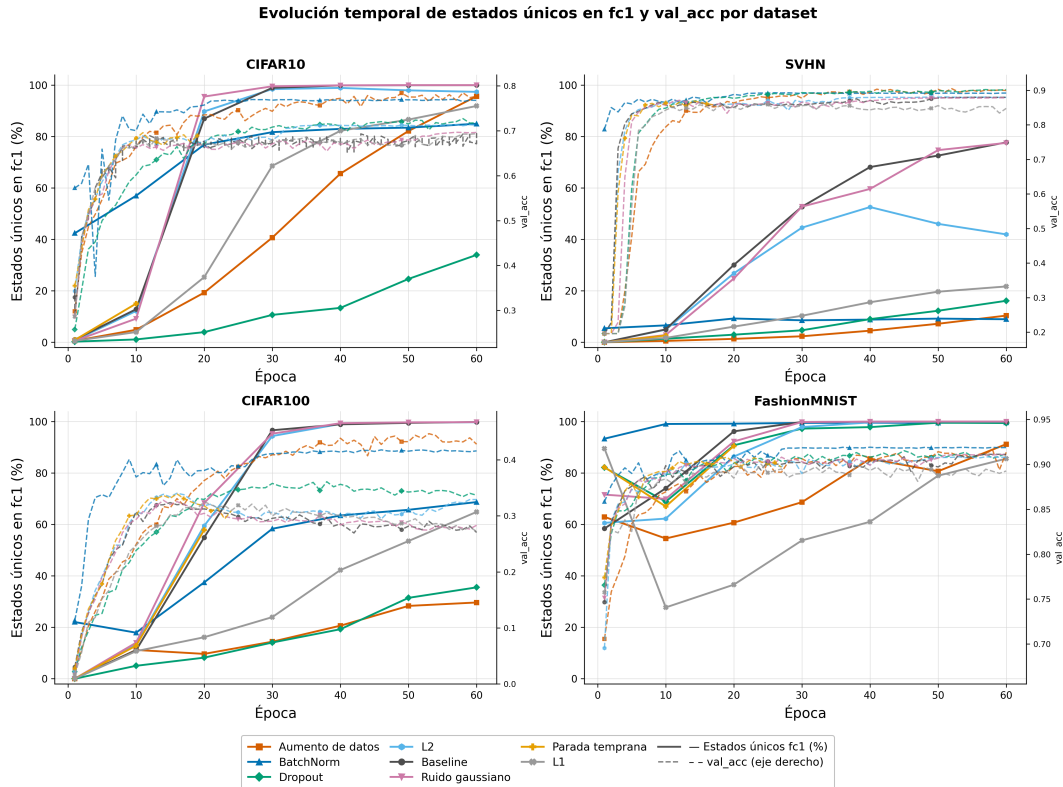


Figure 5.5: Temporal evolution of diversity in `fc1` (left axis, solid lines) and `val_acc` (right axis, dashed lines) by method and dataset.

In all datasets, it is observed that *accuracy* shows a strong increase over the first 10 epochs, being extremely noticeable in SVHN. For example, in this *dataset*, the *baseline* and Gaussian noise reach 90% `val_acc` by epoch 10, while their percentages of unique states continue growing until the end of training. This fact seems to align with these two methods presenting the highest generalization gap for this set (~ 0.12 , see Section 5.1). Conversely, Data Augmentation, which presented the lowest gap in SVHN (~ 0.018), is seen to present a much slower and more controlled growth of unique states, reaching only around 25% of unique states by epoch 60.

For CIFAR-10, certain similar patterns are observed, showing that Data Augmentation achieves excellent *accuracy* along with *Batch normalization*, but with a more sustained evolution in its internal diversity (*Batch normalization* grows little, but starts from $\sim 40\%$ unique states, probably due to the stabilization of the internal activation distributions). In contrast, CIFAR-100 presents very varied and unequal trajectories in its states and accuracy, but the relative order of the methods by internal diversity is similar to the other datasets. Finally, it is difficult to draw clear conclusions from FashionMNIST,

since all methods seem to achieve very high performance, and, although the percentage of states is also generally high for this dataset, no clear relationship with the validation accuracy by method is perceived.

In general, it is observed that accuracy saturates with medium/low values of the percentage of states in a few epochs. Continuing to train seems to complicate the internal state structure without a clear contribution to generalization.

5.2.3 Robustness to Data Noise (EXP-4)

In this experiment, independent additive Gaussian noise at the pixel level is added to the validation images, keeping the weights unchanged, to evaluate the generalization capacity of each model against out-of-distribution distributions. The intensity of the noise is determined by the standard deviation σ of the Gaussian perturbation applied to images normalized in the range $[0, 1]$.

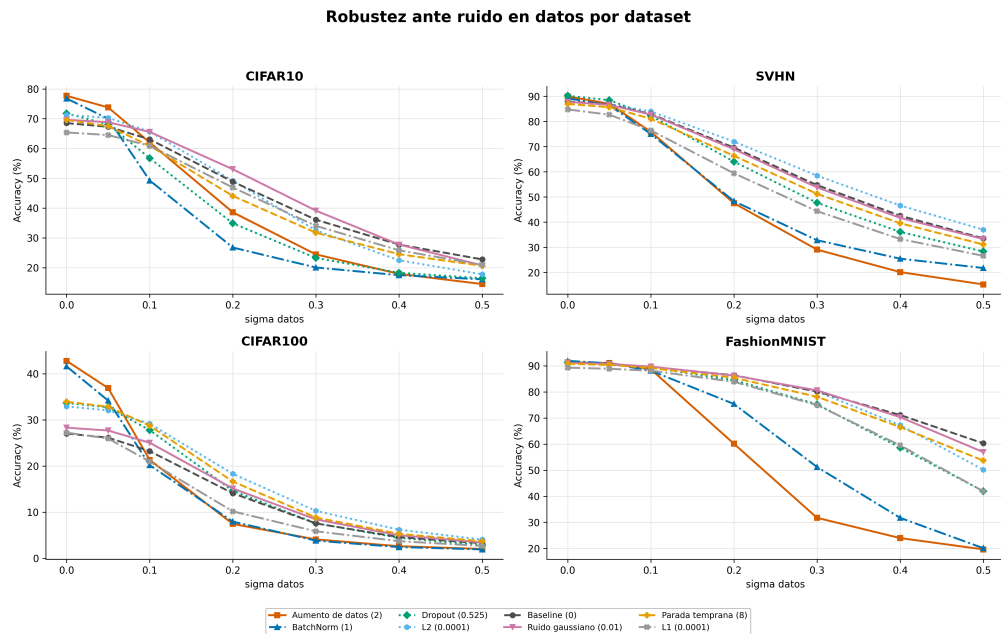


Figure 5.6: Performance curves as a function of Gaussian noise intensity (σ) in the input data for the four domains. The models were trained exclusively with clean data.

The results seem to indicate a **trade-off between performance and robustness**: the methods that lead in precision under clean conditions are the ones that suffer a more severe degradation against intense perturbations. As observed in Figure 5.6, the curves for **Data Augmentation** and **Batch Normalization** drop more steeply than the rest, reaching retention levels of $\sim 18.7\%$ and $\sim 21.0\%$ respectively in CIFAR-10 when reaching the maximum noise ($\sigma = 0.5$, which for an image between 0 and 1 is a lot, as can be visualized in Figure 5.7). This pattern is repeated in SVHN, where both methods fall below 20% retention, while the **Baseline** and **L2** maintain a relative resistance (ratio between validation hits) greater than 30%.

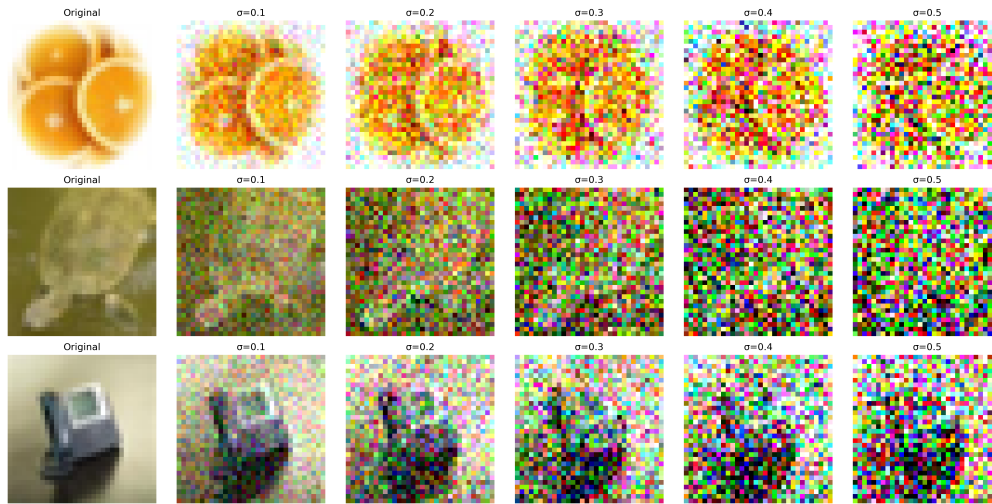


Figure 5.7: Visualization of three validation images of CIFAR-100 with Gaussian noise of increasing intensity, between $\sigma = 0$ (clean) and $\sigma = 0.5$ (noisy).

Table 5.2 summarizes this retention capacity in all domains. Unlike the complete curves shown in Figure 5.6, the tabulated results are presented for a sigma of 0.2, which, according to the previous visualization, has been considered a reasonable compromise between visual degradation and sufficient preservation of the image’s semantic structure.

Unlike what was observed in the base accuracy, the **Baseline** and **L2** regularization present a greater relative resilience, maintaining retention percentages generally greater than 50% in most scenarios. However, this robustness must be interpreted with caution: in complex datasets like **CIFAR-100**, where the absolute accuracy remains low even in clean conditions, a greater sensitivity to noise may indicate that the model has not yet learned sufficiently stable representations of the domain. In this context, the bottleneck seems to be more related to the model’s representational capacity than to a classic overfitting phenomenon.

In **SVHN**, the pattern is analogous: **L2** stands out again with the highest retention (81.9%), followed very closely by the **Baseline** and training with **Gaussian Noise**. In contrast, the methods with the highest precision in clean conditions, such as **BatchNorm** and **Data Augmentation**, once again show a more pronounced relative degradation. This behavior suggests that certain regularization techniques favor specialization on the distributions observed during training, but not necessarily a generic invariance to stochastic out-of-distribution perturbations.

The case of **CIFAR-100** is particularly relevant. Although **BatchNorm** and **Data Augmentation** achieve the highest accuracies on clean data, both methods show extremely pronounced degradation even for moderate noise levels. Rather than indicating only a lack of robustness, this behavior suggests that the model has not yet captured the structure of the domain stably enough: small perturbations significantly alter decision boundaries that were already fragile to begin with. In other words, the problem seems to be dominated by limitations of capacity and representation rather than by an excess of specialization.

Finally, in domains like **FashionMNIST**, although the base accuracy is very high, the degradation induced by the noise still clearly discriminates between techniques: while **Baseline**, **L2**, and **Gaussian Noise** conserve more than 94% of their original performance for $\sigma = 0.2$, methods like **Data Augmentation** present a much steeper drop. Overall, the results reinforce the existence of a trade-off between

Method	Acc. $\sigma=0$	Acc. $\sigma=0.2$	Retention (%)
CIFAR-10			
Gaussian Noise	69.630	53.037	76.2
L1	65.380	46.910	71.7
Baseline	68.530	48.853	71.3
L2	71.250	49.180	69.0
Early Stopping	69.600	44.100	63.4
Data Augmentation	77.680	38.587	49.7
Dropout	71.760	34.890	48.6
BatchNorm	76.770	26.770	34.9
SVHN			
L2	87.953	72.023	81.9
Baseline	87.980	69.626	79.1
Gaussian Noise	87.827	68.983	78.5
Early Stopping	87.000	66.400	76.3
Dropout	90.208	64.061	71.0
L1	84.784	59.413	70.1
BatchNorm	89.213	48.353	54.2
Data Augmentation	89.947	47.545	52.9
CIFAR-100			
L2	32.920	18.300	55.6
Gaussian Noise	28.320	15.140	53.5
Baseline	27.040	14.090	52.1
Early Stopping	33.900	16.600	49.1
Dropout	33.690	14.623	43.4
L1	27.240	10.160	37.3
BatchNorm	41.610	7.940	19.1
Data Augmentation	42.800	7.440	17.4
FashionMNIST			
L2	90.790	86.390	95.2
Baseline	91.100	86.383	94.8
Gaussian Noise	91.100	86.257	94.7
Early Stopping	90.800	85.400	94.1
L1	89.260	83.857	93.9
Dropout	91.180	84.483	92.7
BatchNorm	91.870	75.373	82.0
Data Augmentation	91.340	60.113	65.8

Table 5.2: Retention of *accuracy* against Gaussian data noise ($\sigma=0.2$) by method in the four evaluated datasets.

maximum precision and stability against perturbations, although this relationship depends strongly on the intrinsic complexity of the domain and the model’s effective capacity to represent it.

5.2.4 Robustness to Weight Noise and Flat Minima Analysis (EXP-5)

Gaussian noise is added to the weights of specific layers after training to measure the parametric stability of each model. This experiment evaluates the curvature of the minimum to which each regularizer converges: a high performance retention against perturbation indicates the presence of a “flat” minimum (*flat minimum*). The perturbation level used, $\sigma = 0.15$, was selected as a balance point between being sufficiently informative —small perturbations do not discriminate between methods— and not being trivially destructive for all models. It is important to note that this value is **absolute**: the same standard deviation is added to all weights regardless of their magnitude. The practical implication is that the real impact of the perturbation depends on the scale of the weights in each layer, an aspect that is revisited in the subsequent analysis.

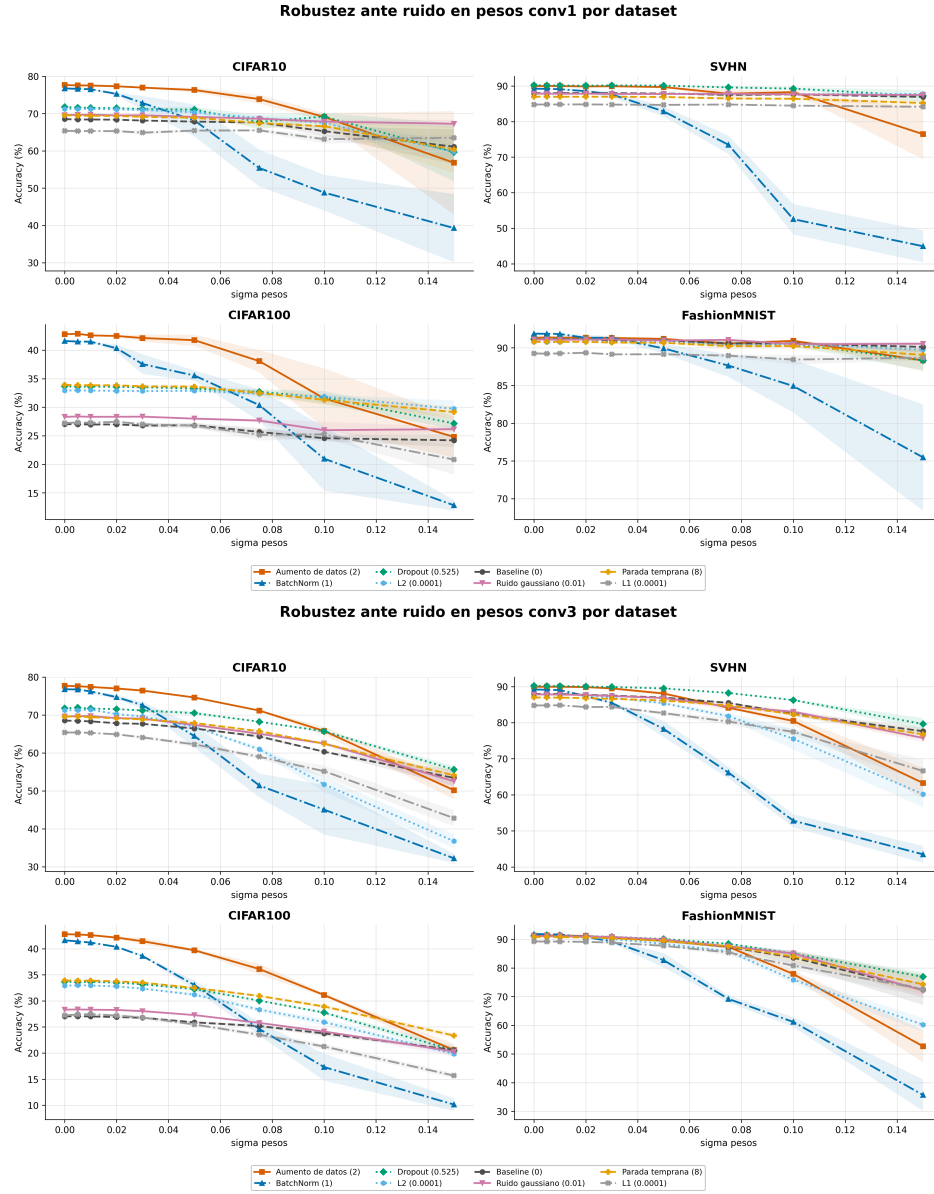


Figure 5.8: Accuracy degradation curves against noise in the weights of the convolutional layers across the four domains. Top: perturbation in `conv1`. Bottom: perturbation in `conv3`.

The most striking finding is the behavior of **BatchNorm**: despite its high performance in clean conditions, it collapses drastically against perturbations in the weights of the convolutional layers. This pattern, observed in Figure 5.8 for all datasets, suggests that batch normalization tends to converge to “sharp” minima (*sharp minima*) in the feature extraction layers.

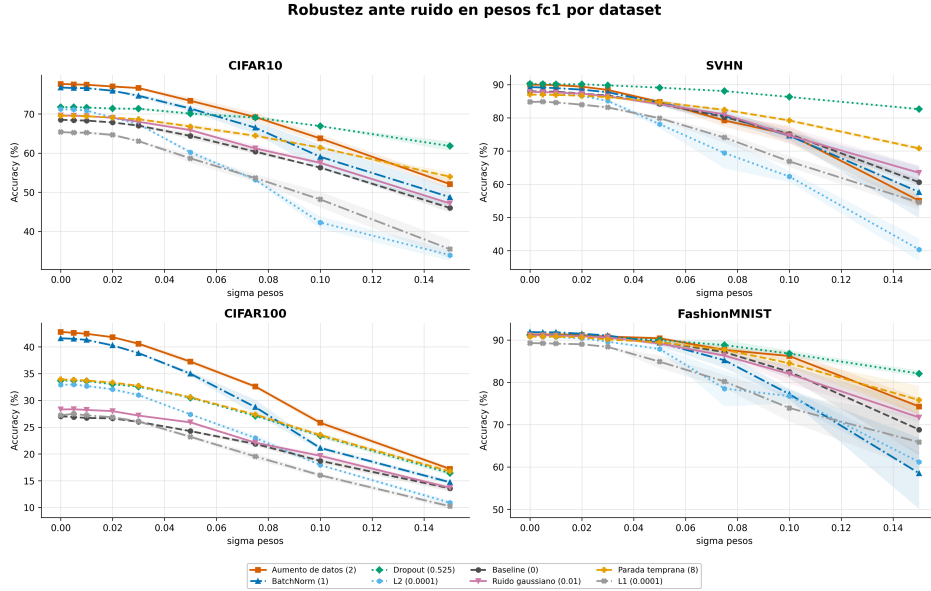


Figure 5.9: Accuracy degradation curves when perturbing the fully connected layer `fc1` in the four domains. **Dropout** stands out for showing the highest parametric robustness.

Figure 5.9 captures the behavior against perturbations in `fc1`. Unlike the convolutional layers, here **Dropout** clearly emerges as the most robust method in all domains, maintaining high retentions even for the highest levels of σ tested. **BatchNorm**, on the other hand, partially recovers its stability compared to `conv3`, which points to the fact that its fragility is particularly concentrated in the feature extraction layers and does not uniformly extend to the classification layers.

Table 5.3 summarizes the retention percentages for a noise level of $\sigma = 0.15$. The data confirm that **Dropout** is the only method with high and consistent robustness across all layers of the architecture. Conversely, **L2** shows a heterogeneous profile: it maintains good resilience in the input layers (`conv1`) but suffers a steep collapse as it advances towards the deeper layers (`conv3` and `fc1`).

Figure 5.10 visually synthesizes this inter-layer heterogeneity. It is observed how the protection offered by regularizers like L2 or L1 degrades significantly in the depth of the network, while Dropout preserves uniform stability.

One possible explanation for the pattern of higher tolerance in early layers is precisely the absolute nature of the applied noise: the initial convolutional layers tend to learn filters with weights of greater magnitude than `fc1`, so $\sigma = 0.15$ represents a smaller relative perturbation.

In this sense, the ratio $\tilde{\sigma}_l = \sigma / \bar{\sigma}_w^{(l)}$ could be defined, where $\bar{\sigma}_w^{(l)}$ is the empirical standard deviation of the weights in layer l . This ratio would normalize the perturbation with respect to the actual scale of each layer and would allow reinterpreting the results of Table 5.3 without modifying any experiment.

This transversal pattern across the four domains confirms that the choice of regularizer not only dictates the final precision but also defines the geometric nature of the convergence point in the loss landscape.

Method	Ret. conv1 (%)	Ret. conv3 (%)	Ret. fc1 (%)
CIFAR-10			
Dropout	83.1	77.5	86.1
Early Stopping	86.8	77.7	77.6
Gaussian Noise	96.6	75.5	67.7
Baseline	89.2	77.9	67.1
Data Augmentation	73.1	64.6	67.0
BatchNorm	51.2	42.0	63.5
L1	97.2	65.5	54.2
L2	83.9	51.6	47.6
SVHN			
Dropout	96.7	88.3	91.6
Early Stopping	98.0	88.2	81.4
Gaussian Noise	99.6	86.3	72.2
Baseline	98.8	88.2	68.9
BatchNorm	50.4	48.8	64.6
L1	99.3	78.6	64.3
Data Augmentation	85.0	70.3	61.2
L2	99.8	68.4	45.9
CIFAR-100			
Baseline	89.5	76.2	50.2
Early Stopping	86.0	68.9	49.4
Dropout	80.6	60.9	48.7
Gaussian Noise	92.4	71.7	48.3
Data Augmentation	58.0	48.1	40.2
L1	76.5	57.6	37.5
BatchNorm	30.8	24.4	35.4
L2	90.3	60.3	33.1
FashionMNIST			
Dropout	96.9	84.4	90.0
Early Stopping	98.1	81.8	83.5
Data Augmentation	96.7	57.7	81.3
Gaussian Noise	99.4	79.2	78.7
Baseline	98.9	79.5	75.6
L1	99.4	81.3	73.8
L2	98.9	66.3	67.4
BatchNorm	82.2	38.9	63.7

Table 5.3: Retention of *accuracy* against weight noise ($\sigma=0.15$) by layer and method in the four evaluated datasets.

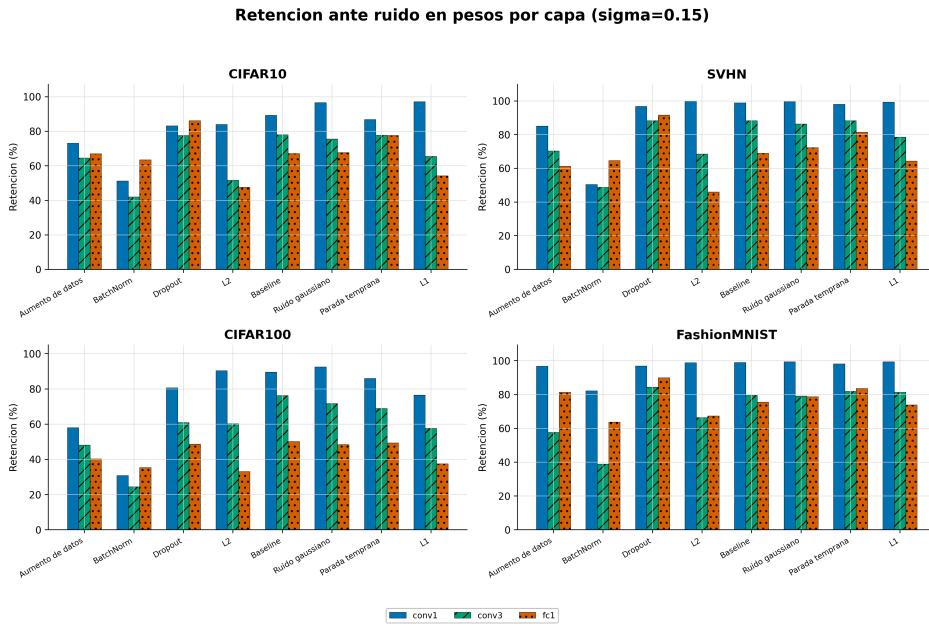


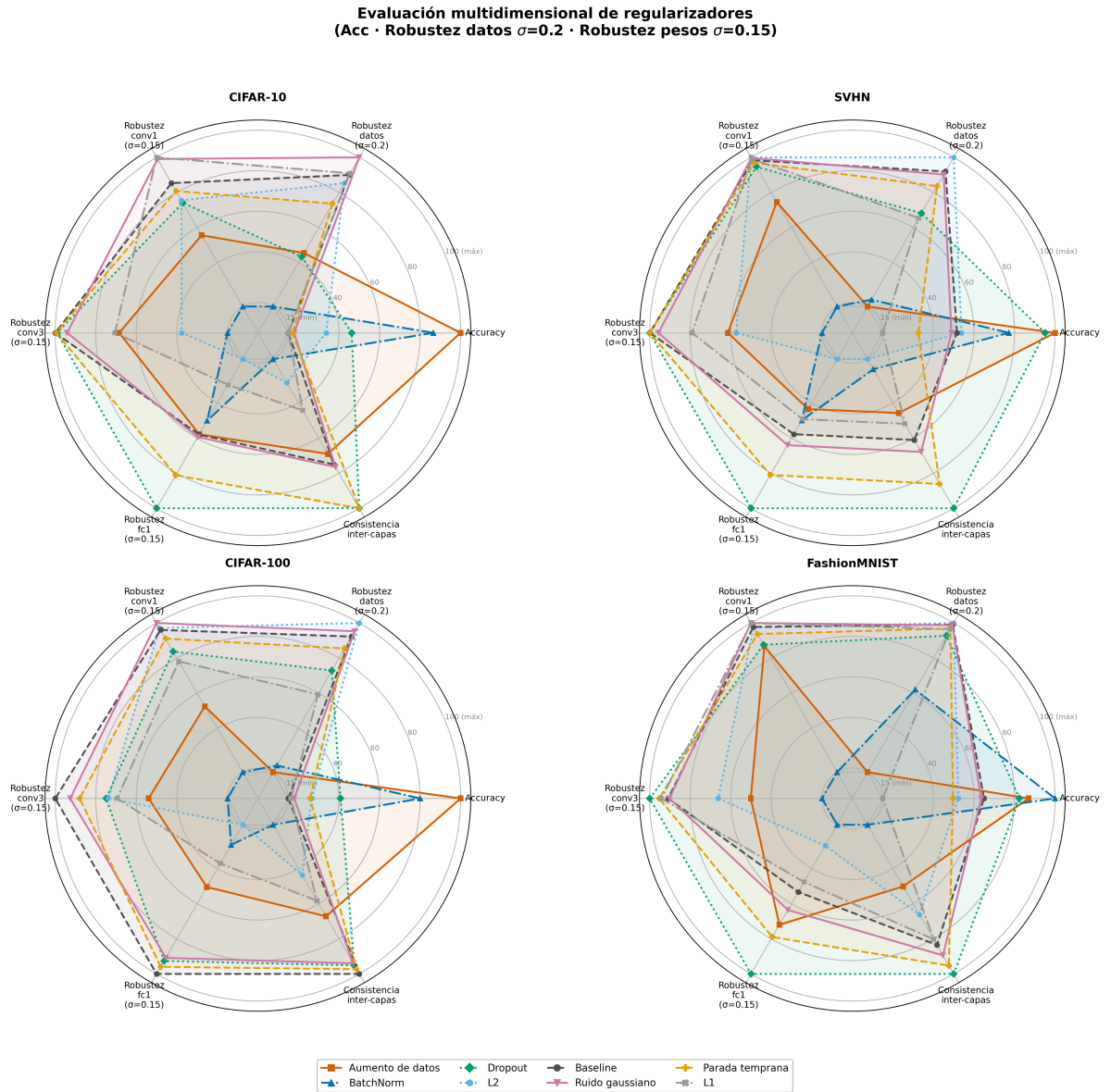
Figure 5.10: Summary of retention at $\sigma = 0.15$ for each method and layer in all datasets. The consistency of **Dropout** contrasts with the fragility of **BatchNorm** and the heterogeneity of **L2**.

5.3 Discussion

The five previous experiments provide results from independent perspectives: comparative *accuracy*, internal representations, temporal evolution, robustness to data noise, and parametric stability. Next, these results are connected with each other and with the existing literature.

The most transversal result of the study is that no method dominates in all relevant dimensions. If validation *accuracy* were the only criterion, the answer would be trivial: use Data Augmentation. However, the robustness experiments reveal a considerably more complex picture. Data Augmentation, the most precise method, is one of the least robust to pixel noise (which is not necessarily negative, since “natural noise” is of a different type); Batch Normalization converges to sharp minima in convolutional layers and collapses under moderate weight perturbations; techniques like L2 or the Baseline show surprising robustness to noisy inputs but sacrifice performance; and Dropout is the only one with consistent robustness in all layers, converging to flat minima in line with the literature [7, 8].

Figure 5.11 synthesizes this extensive heterogeneity across the four evaluated domains. The direct practical implication is that the choice of regularizer should be guided by the problem’s priorities, not by a single performance indicator.



Normalización: min-max por dimensión con suelo = 15 (peor método observado) y punta = 100 (mejor). Preserva magnitud de diferencias entre métodos.

Figure 5.11: Radar charts showing the profile of each regularizer in the four datasets. Each axis represents a different evaluation dimension: validation accuracy, robustness to data noise ($\sigma=0.20$), and robustness to weight noise per layer (conv1 , conv3 , fc1 , $\sigma=0.15$). Values are normalized per dimension using min-max with a floor at 15, so that the best observed method reaches 100 and the worst 15, preserving the real relative distances between methods. It is observed that no regularizer simultaneously maximizes all dimensions. Inter-layer consistency is defined as $\max(\text{conv1}, \text{conv3}, \text{fc1}) - \min(\text{conv1}, \text{conv3}, \text{fc1})$ over its weight robustness metric.

This variety among methods is no coincidence. The internal activations experiments seem to show that *unique_pctg*, entropy, and dispersion ratio in $\mathbf{f}_{c1}$ influence performance, with an intermediate representational complexity zone (an acceptable range) where the highest *accuracy* methods are concentrated; collapsed or saturated methods are simultaneously displaced to the extremes. This zone is interpretable: below it, the network loses discriminative power, and above it, it tends to memorize the training. These signals are detectable without waiting for the final result, opening the door to early model diagnostics.

The temporal evolution experiment adds a dynamic dimension to the analysis. In most methods, a relationship is observed between sustained growth in the percentage of unique states and a low generalization gap. This behavior is consistent with the *information bottleneck* interpretation proposed by [32], where neural networks go through an initial phase of rapid adjustment to the relevant signal followed by a compression of redundant or less useful information for the task.

However, some limitations must be pointed out. The employed architecture is small and the conclusions should not be directly extrapolated to deeper models. The *unique_pctg* metric depends on the discretization resolution, so the trends are qualitative, although its relationship with the other metrics and its consistency in the relationship between methods for different resolutions indicate that it is a valid metric. The experiments study techniques in isolation, ignoring possible non-linear effects in their combinations [33]. Finally, the number of tests conducted forces the results to be interpreted as robust (but empirical) trends rather than as exact conclusions.

CONCLUSIONS AND FUTURE WORK

6.1 Conclusions

In this work, the internal effect of seven regularization techniques on a standard CNN architecture has been analyzed, adapting the training process to record the evolution of the representations in each layer. Furthermore, the external evaluation of *accuracy* has been complemented with five experiments focused on the internal behavior of the model. Together, the results help us answer the questions posed at the beginning.

The main result is that internal activation metrics can indeed be related to some extent with model performance and help explain phenomena regarding its behavior such as overfitting, robustness to new data, or when to stop training. The percentage of unique states, Shannon entropy, and the dispersion ratio in f_{c1} covary positively with each other, suggesting that they reflect the same underlying property: the informational diversity of the learned representations, and that they are a valid measure for this phenomenon (see the figures with their relationships in Appendices). The best performing methods are located in an intermediate zone in this diversity, and tend to present a sustained growth throughout training in their internal activation dispersion metrics: below it, the network loses discriminative capacity, while above it tends to memorize instead of generalize. This pattern tends to be maintained across different *datasets* and is supported by the observed temporal dynamics, where all methods go through an initial phase of high learning before progressively differentiating [32].

The result that best justifies the work is that validation *accuracy* is not enough to characterize regularization. The robustness experiments show very different profiles between methods with similar performance. Data Augmentation, the most accurate one, learns invariances linked to the type of transformation applied during training, which translates into fragility against perturbations of another nature. Batch Normalization accelerates and stabilizes optimization with great efficiency, but converges to sharp minima in convolutional layers and turns out to be the most vulnerable to perturbations in the weights. *Dropout*, despite its lower *accuracy*, systematically converges to flat minima in all layers, in line with the literature [7, 8].

From this, the central conclusion of the work is drawn: there is no universally superior regularization technique. The optimal choice depends on the priorities of the problem: *Data Augmentation* if performance in clean conditions is sought, *Dropout* if parametric stability is required, or methods like L2 and the Baseline if greater resilience against stochastic input noise is sought. Rather than looking for a «winning» method, it seems more useful to first characterize the needs of the problem and validate later, relying on internal metrics, that the chosen technique acts as expected.

6.2 Future Work

After conducting this study, several ways to continue are observed, both direct extensions of the experiments and more ambitious methodological lines.

6.2.1 Experimental extensions

The first thing that could be done is to test the same protocol on deeper architectures (ResNet-18, VGG-16), to study whether the optimal ranges of unique states, entropy and dispersion ratio observed in this work also shift depending on the network used and not only on the *dataset*. Likewise, it would be relevant to extend the analysis to more complex datasets such as ImageNet, or even to domains of a different nature, such as text recognition, with the aim of evaluating whether the observed relationships between internal metrics and generalization are maintained outside the vision domain.

A second possible extension is the analysis of combinations of regularizers. This work studies each technique in isolation; in practice, models combine multiple techniques with non-trivial interactions [33], and it may be relevant to study whether activation metrics remain diagnostic in the face of multiple regularization. For instance, the combination of *Gaussian Noise* with *Data Augmentation* could result in high performance with better endurance against noise or imperfections in validation data.

Furthermore, the similarity between representations learned by different regularizers could be studied using **CKA (Centered Kernel Alignment)** [34], a metric that allows quantifying whether two techniques produce structurally equivalent representation spaces.

6.2.2 Methodological lines

One possible idea would be to add distance metrics between training and validation distributions (KL divergence, EMD distance) throughout training to anticipate overfitting before it manifests in the *accuracy* curves.

Another alternative route is the development of an automatic diagnostic *framework* that, given a trained model, calculates internal activation metrics and issues a warning signal about possible under- or over-regularization. This line connects with the emerging area of interpretability, where recent works [35] explore the use of activation entropy as an explicit constraint in the loss function, which points out that the proposed methodological line has active projection in contemporary research.

BIBLIOGRAPHY

- [1] C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals, "Understanding deep learning requires rethinking generalization," in *International Conference on Learning Representations (ICLR)*, 2017.
- [2] I. Nusrat and S.-B. Jang, "A comparison of regularization techniques in deep neural networks," *Symmetry*, vol. 10, no. 11, p. 648, 2018.
- [3] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [4] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, "mixup: Beyond empirical risk minimization," in *International Conference on Learning Representations*, 2018.
- [5] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.
- [6] J. Tompson, R. Goroshin, A. Jain, Y. LeCun, and C. Bregler, "Efficient object localization using convolutional networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 648–656, 2015.
- [7] S. Hochreiter and J. Schmidhuber, "Flat minima," *Neural Computation*, vol. 9, no. 1, pp. 1–42, 1997.
- [8] N. S. Keskar, D. Mudigere, J. Nocedal, M. Smelyanskiy, and P. T. P. Tang, "On large-batch training for deep learning: Generalization gap and sharp minima," in *International Conference on Learning Representations (ICLR)*, 2017. arXiv:1609.04836.
- [9] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *International Conference on Machine Learning (ICML)*, 2015.
- [10] D. J. MacKay, *Information Theory, Inference, and Learning Algorithms*. Cambridge university press, 2003.
- [11] V. N. Vapnik and A. Y. Chervonenkis, "On the uniform convergence of relative frequencies of events to their probabilities," *Theory of Probability & Its Applications*, vol. 16, no. 2, 1971.
- [12] M. Mohri, A. Rostamizadeh, and A. Talwalkar, *Foundations of Machine Learning*. MIT Press, 2012.
- [13] C. F. G. d. Santos and J. P. Papa, "Avoiding overfitting: A survey on regularization methods for convolutional neural networks," *ACM Computing Surveys*, vol. 54, no. 10s, pp. 1–25, 2022.
- [14] R. Moradi, R. Berangi, and M. Minaei, "A survey of regularization strategies for deep models," *Artificial Intelligence Review*, vol. 53, no. 6, pp. 3947–3986, 2020.
- [15] S. Deepa, R. Siddalingappa, P. Kalpana, J. Loveline Zeema, M. Vinay, J. Jayapriya, J. Suganthi, and I. Priya Stella Mary, "A comparative analysis of L1, L2, and L1L2 regularization techniques in neural networks for image classification," *International Journal of Engineering Trends and Technology*, vol. 73, no. 10, pp. 107–116, 2025.
- [16] K. Mohamed, I. Suliman, and M. Alfeel, "A modified high-order neural network with smoothing L1 regularization and momentum terms," *Signal, Image and Video Processing*, 2025.
- [17] F. A. Mejía Cajicá, J. A. García Henao, C. J. Barrios Hernández, and M. Riveill, "Analysis of regularization in deep learning models on testbed architectures," in *High Performance Computing. CARLA 2020*, vol. 1327 of *Communications in Computer and Information Science*, Springer, 2021.
- [18] M. Amer and T. Maul, "A review of modularization techniques in artificial neural networks," *Artificial Intelligence Review*, 2019.
- [19] S. Santurkar, D. Tsipras, A. Ilyas, and A. Madry, "How does batch normalization help optimization?," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

- [20] B. C. Brown, J. Juravsky, A. Gilboa, and G. Loaiza-Ganem, "Relating regularization and generalization through the intrinsic dimension of activations," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2211.13239.
- [21] C. Liu, G. Christopoulos, G. Pita-Gil, and S. Krishnaswamy, "Assessing neural network representations during training using noise-resilient diffusion spectral entropy," in *ICML 2023 Workshop on Topology, Algebra and Geometry in Machine Learning*, 2023. Also published in IEEE CISS 2024. arXiv:2312.04823.
- [22] Z. Peng *et al.*, "Learning compact representations via intrinsic dimension regularization," in *International Conference on Learning Representations (ICLR)*, 2024.
- [23] A. Krizhevsky, "Learning multiple layers of features from tiny images," tech. rep., University of Toronto, 2009.
- [24] Y. Netzer, T. Wang, A. Coates, A. Bissacco, B. Wu, and A. Y. Ng, "Reading digits in natural images with unsupervised feature learning," in *NIPS Workshop on Deep Learning and Unsupervised Feature Learning*, 2011.
- [25] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms," *arXiv preprint arXiv:1708.07747*, 2017.
- [26] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015.
- [27] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, *et al.*, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32*, pp. 8024–8035, 2019.
- [28] J. D. Hunter, "Matplotlib: A 2d graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.
- [29] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.
- [30] T. pandas development team, "pandas-dev/pandas: Pandas," Feb. 2020.
- [31] C. O. da Costa-Luis, "tqdm: A fast, extensible progress bar for Python and CLI," *Journal of Open Source Software*, vol. 4, no. 37, p. 1277, 2019.
- [32] N. Tishby and R. Schwartz-Ziv, "Opening the black box of deep neural networks via information," in *International Conference on Learning Representations (ICLR) Workshop*, 2017.
- [33] X. Li, S. Chen, X. Hu, and J. Yang, "Understanding the disharmony between dropout and batch normalization by variance shift," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [34] S. Kornblith, M. Norouzi, H. Lee, and G. Hinton, "Similarity of neural network representations revisited," in *International Conference on Machine Learning (ICML)*, pp. 3519–3529, PMLR, 2019.
- [35] Z. Kang *et al.*, "Entropy regularizing activation: Boosting continuous control, large language models, and image classification with activation as entropy constraints," 2025. arXiv:2510.08549.

APPENDICES

ADDITIONAL ACTIVATION HISTOGRAMS

A.1 CIFAR-10

BatchNorm = 1.0 | Dataset: CIFAR10

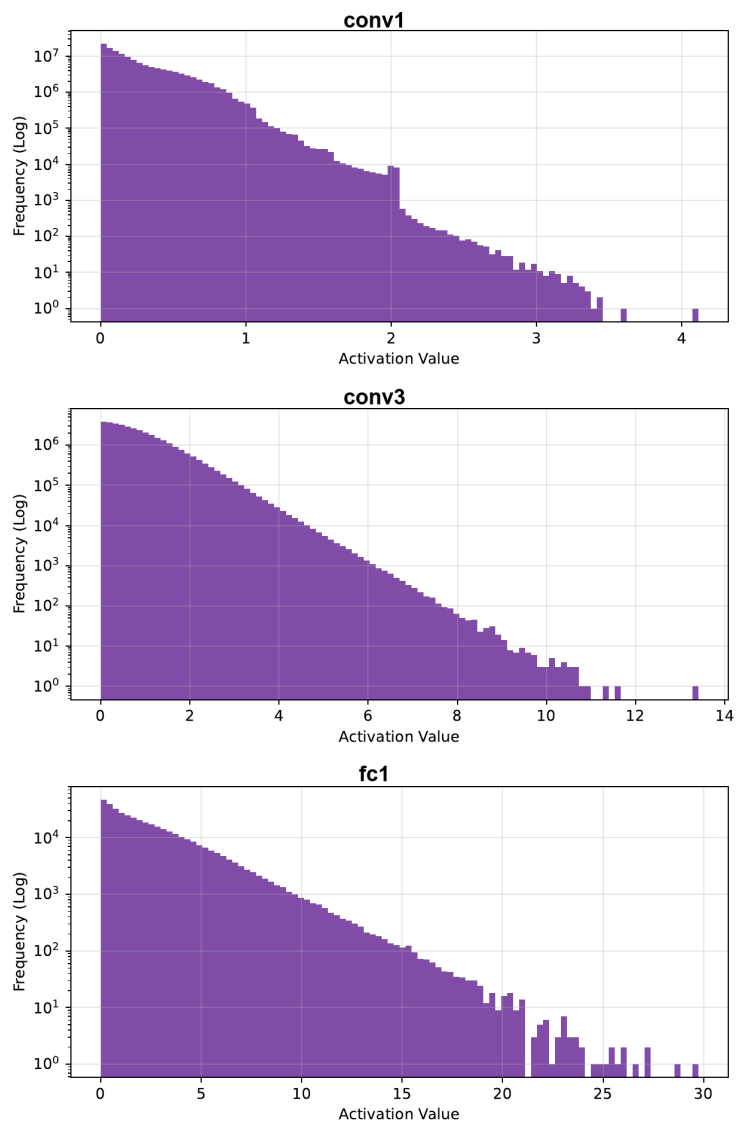


Figure A.1: Distribution of activations per layer (`conv1`, `conv3` and `fc1`), on the validation set of *CIFAR10*, with regularization method BatchNorm and parameter 1.0.

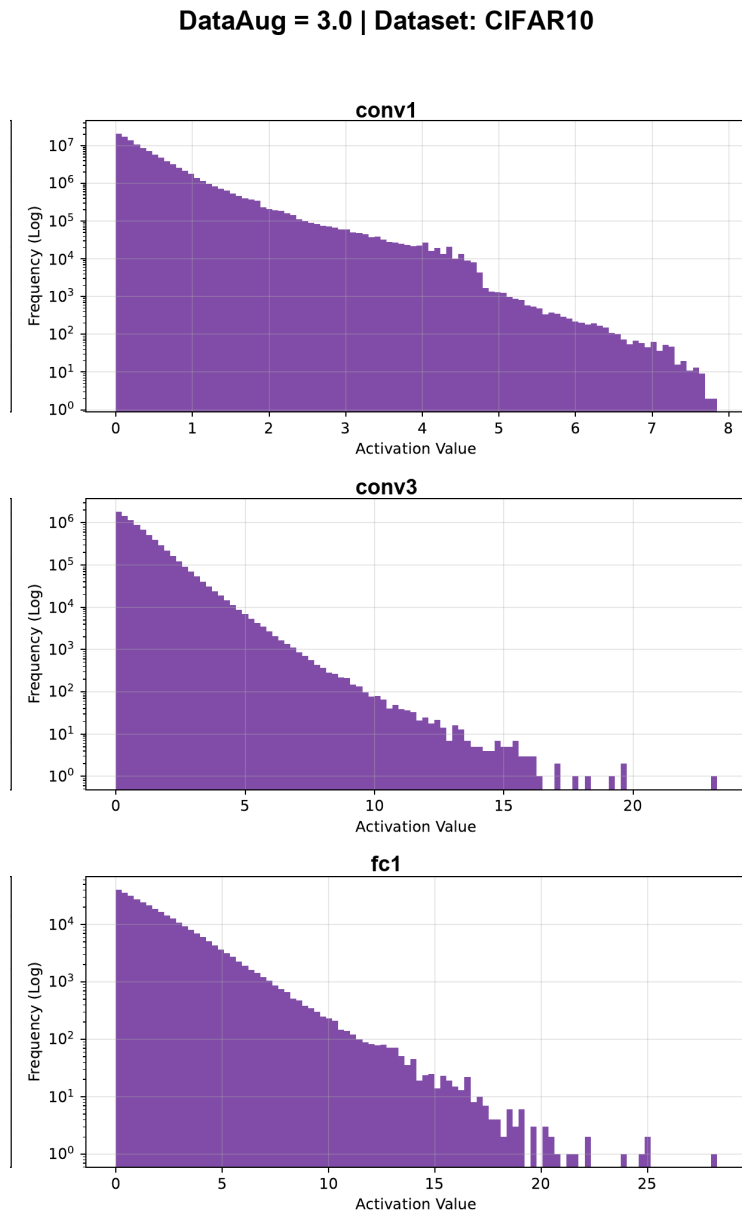


Figure A.2: Distribution of activations per layer (`conv1`, `conv3` and `fc1`), on the validation set of *CIFAR10*, with regularization method DataAug and parameter 3.0.

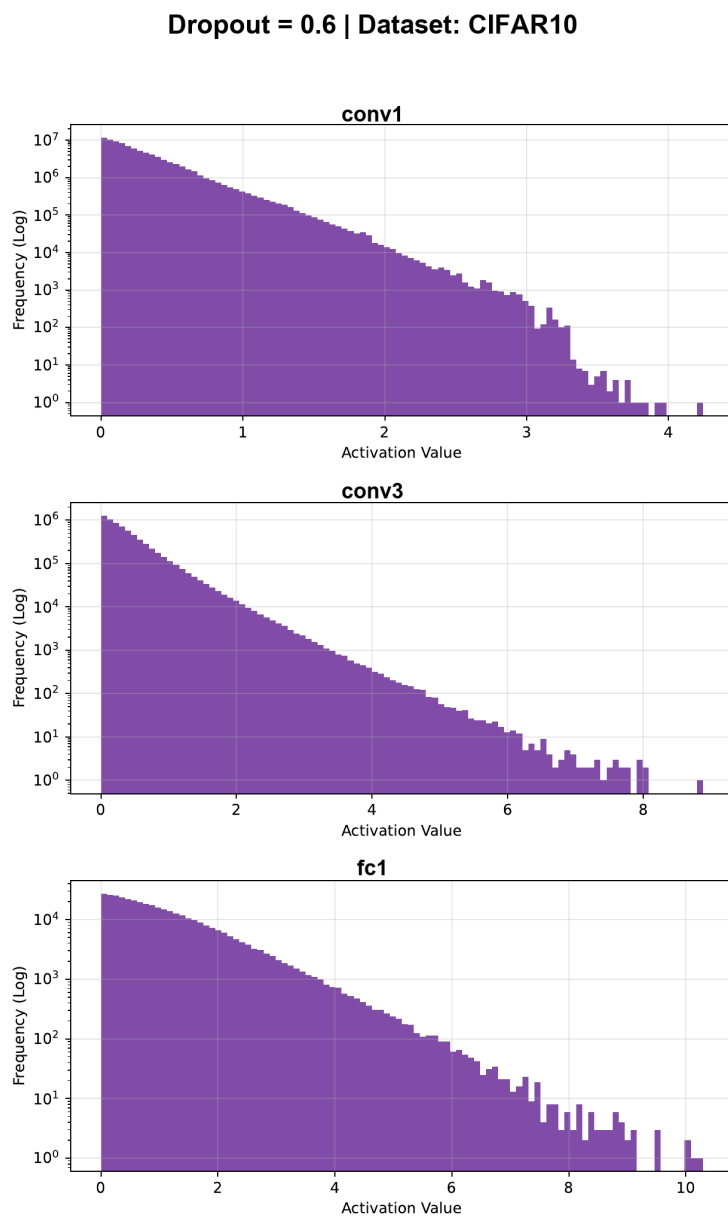


Figure A.3: Distribution of activations per layer (`conv1`, `conv3` and `fc1`), on the validation set of *CIFAR10*, with regularization method Dropout and parameter 0.6.

L2 = 0.01 | Dataset: CIFAR10

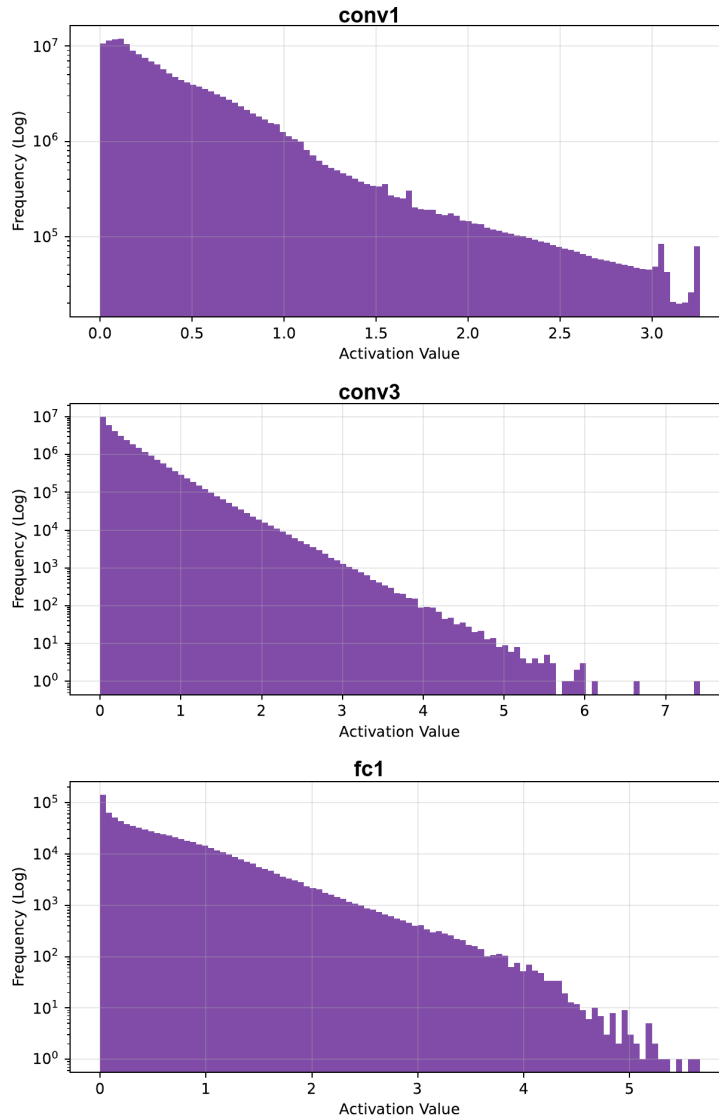


Figure A.4: Distribution of activations per layer (`conv1`, `conv3` and `fc1`), on the validation set of *CIFAR10*, with regularization method L2 and parameter 0.01.

A.2 SVHN

BatchNorm = 1.0 | Dataset: SVHN

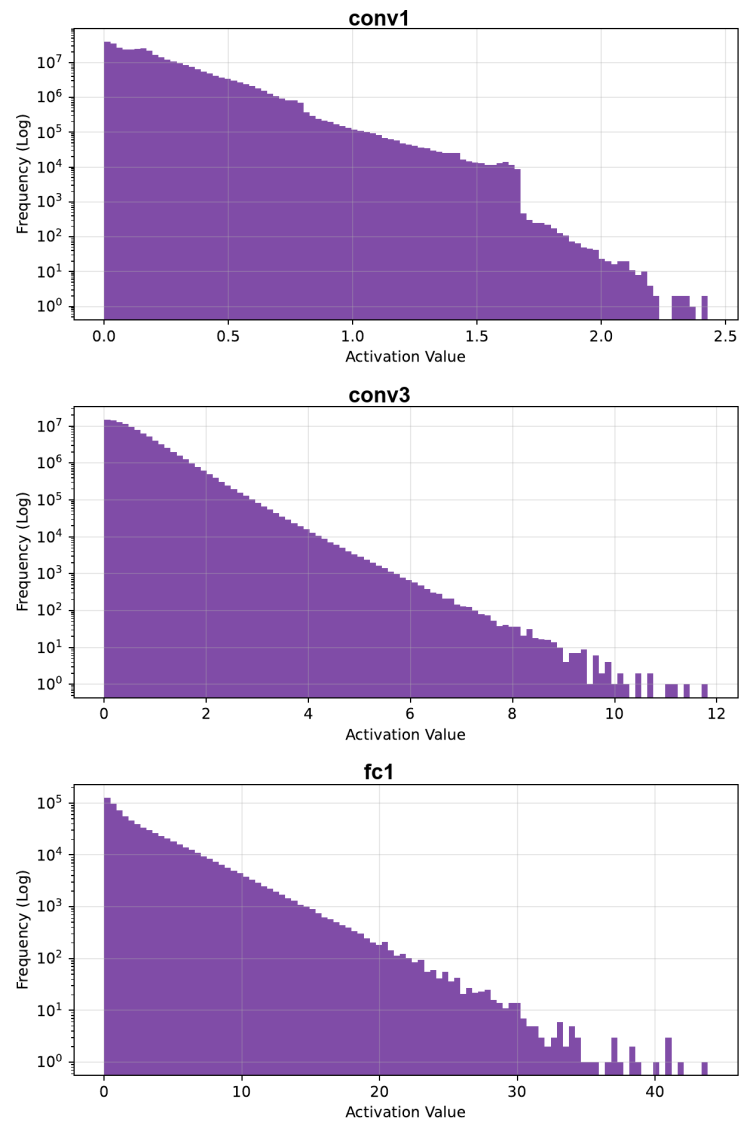


Figure A.5: Distribution of activations per layer (`conv1`, `conv3` and `fc1`), on the validation set of *SVHN*, with regularization method BatchNorm and parameter 1.0.

Dropout = 0.6 | Dataset: SVHN

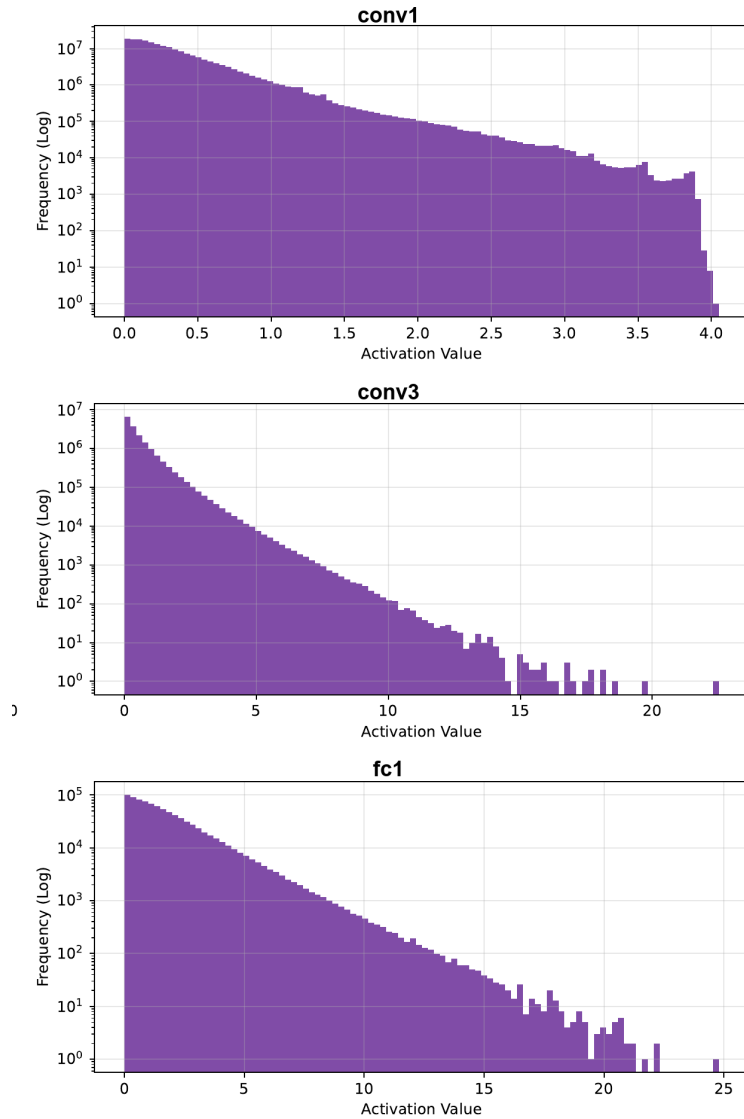


Figure A.6: Distribution of activations per layer (`conv1`, `conv3` and `fc1`), on the validation set of *SVHN*, with regularization method Dropout and parameter 0.6.

L1 = 0.01 | Dataset: SVHN

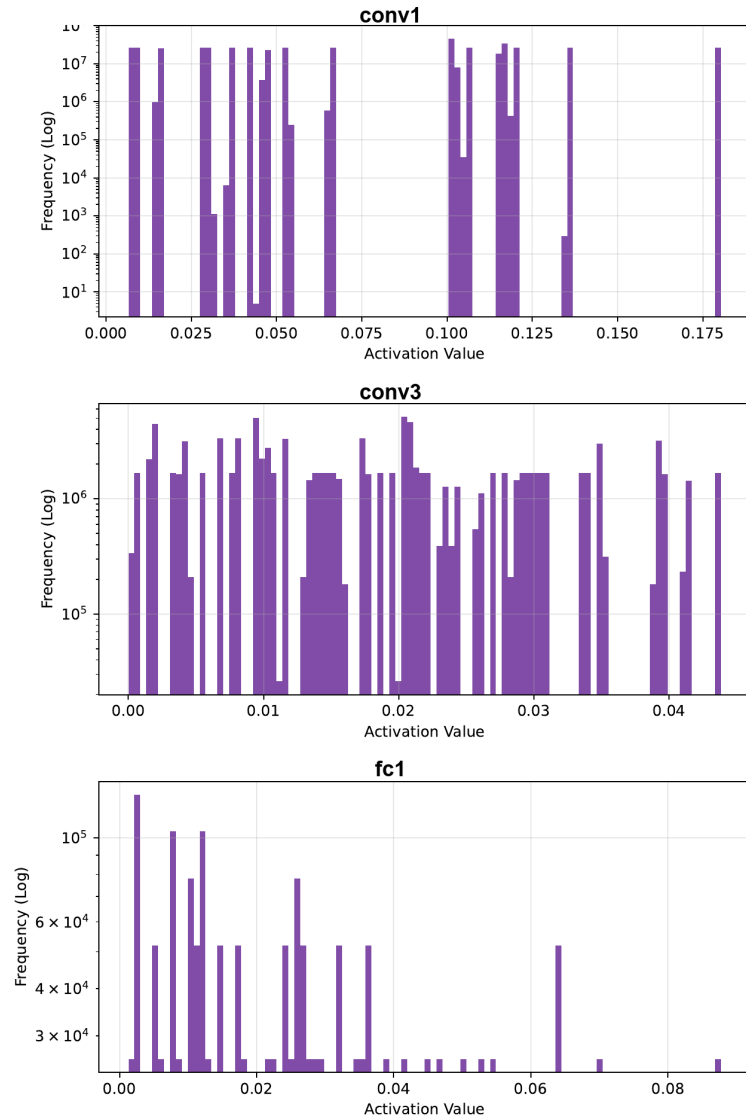


Figure A.7: Distribution of activations per layer (`conv1`, `conv3` and `fc1`), on the validation set of *SVHN*, with regularization method L1 and parameter 0.01.

A.3 CIFAR-100

BatchNorm = 1.0 | Dataset: CIFAR-100

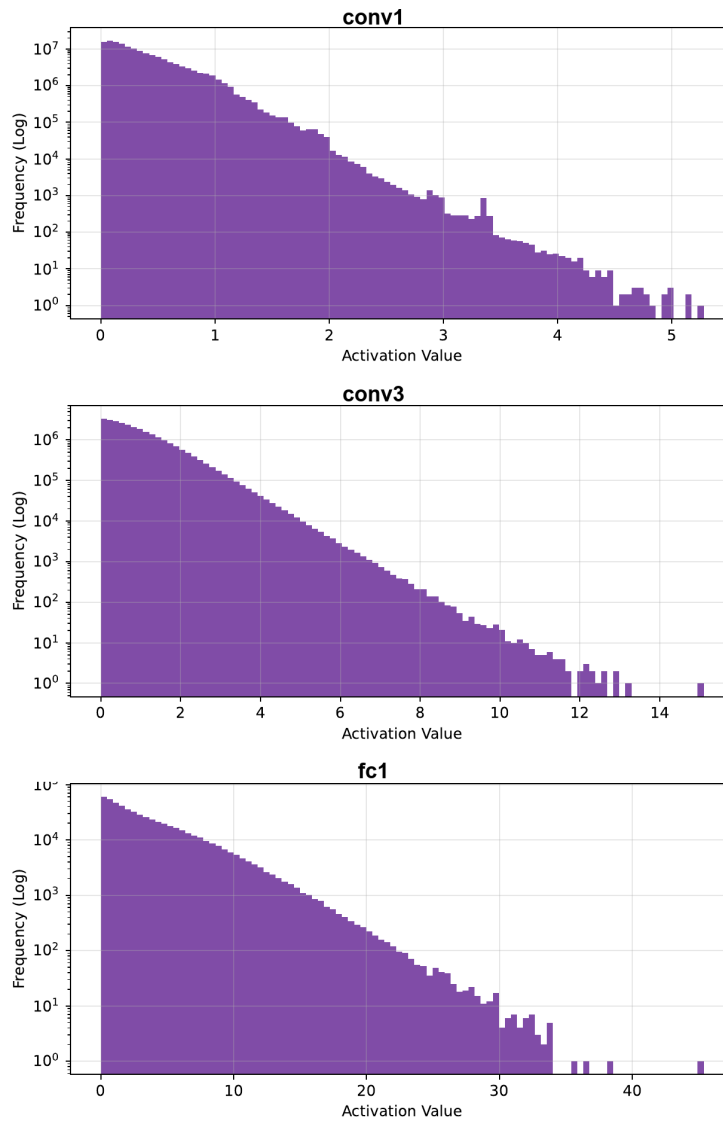


Figure A.8: Distribution of activations per layer (*conv1*, *conv3* and *fc1*), on the validation set of *CIFAR100*, with regularization method BatchNorm and parameter 1.0.

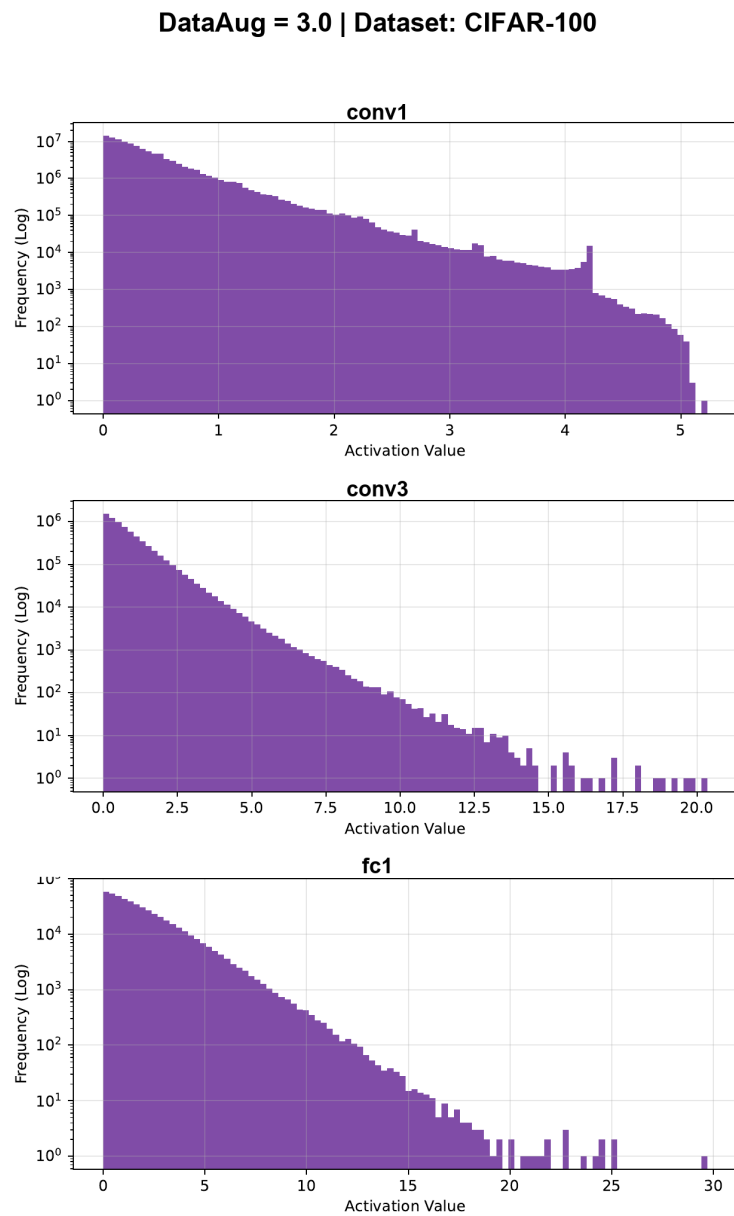


Figure A.9: Distribution of activations per layer (`conv1`, `conv3` and `fc1`), on the validation set of *CIFAR100*, with regularization method DataAug and parameter 3.0.

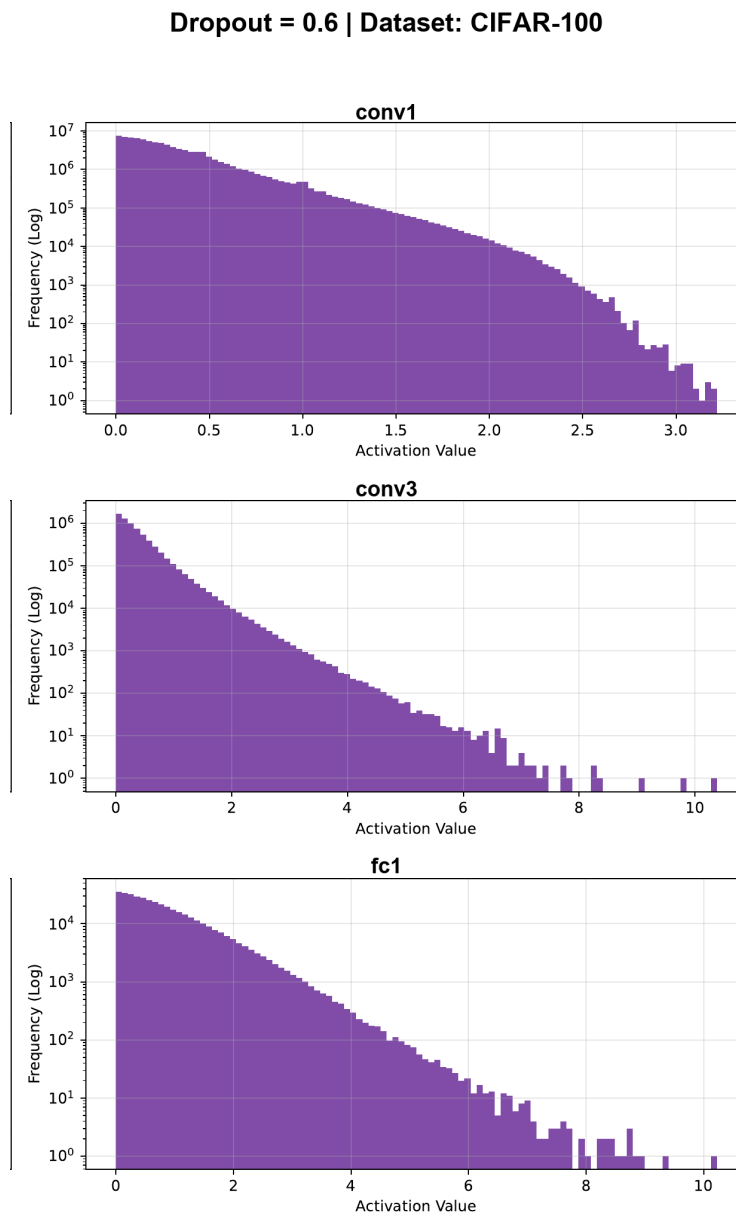


Figure A.10: Distribution of activations per layer (*conv1*, *conv3* and *fc1*), on the validation set of *CIFAR100*, with regularization method Dropout and parameter 0.6.

L1 = 0.01 | Dataset: CIFAR-100

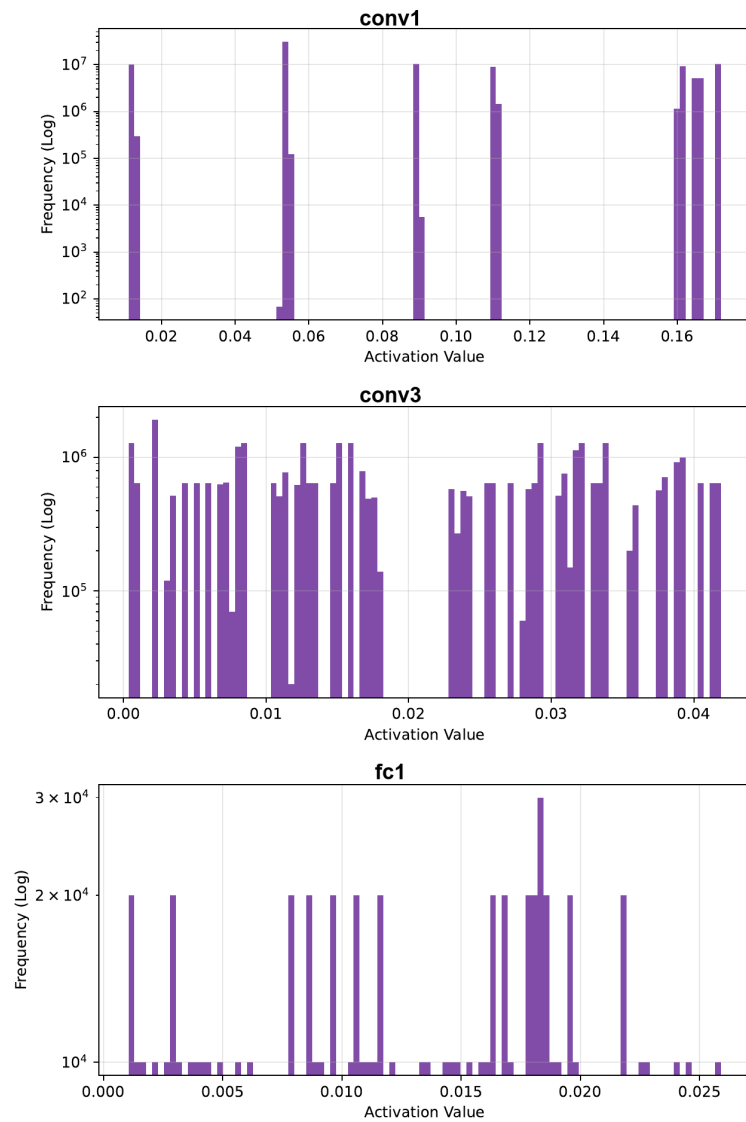


Figure A.11: Distribution of activations per layer (`conv1`, `conv3` and `fc1`), on the validation set of *CIFAR100*, with regularization method L1 and parameter 0.01.

A.4 FashionMNIST

BatchNorm = 1.0 | Dataset: Fashion-MNIST

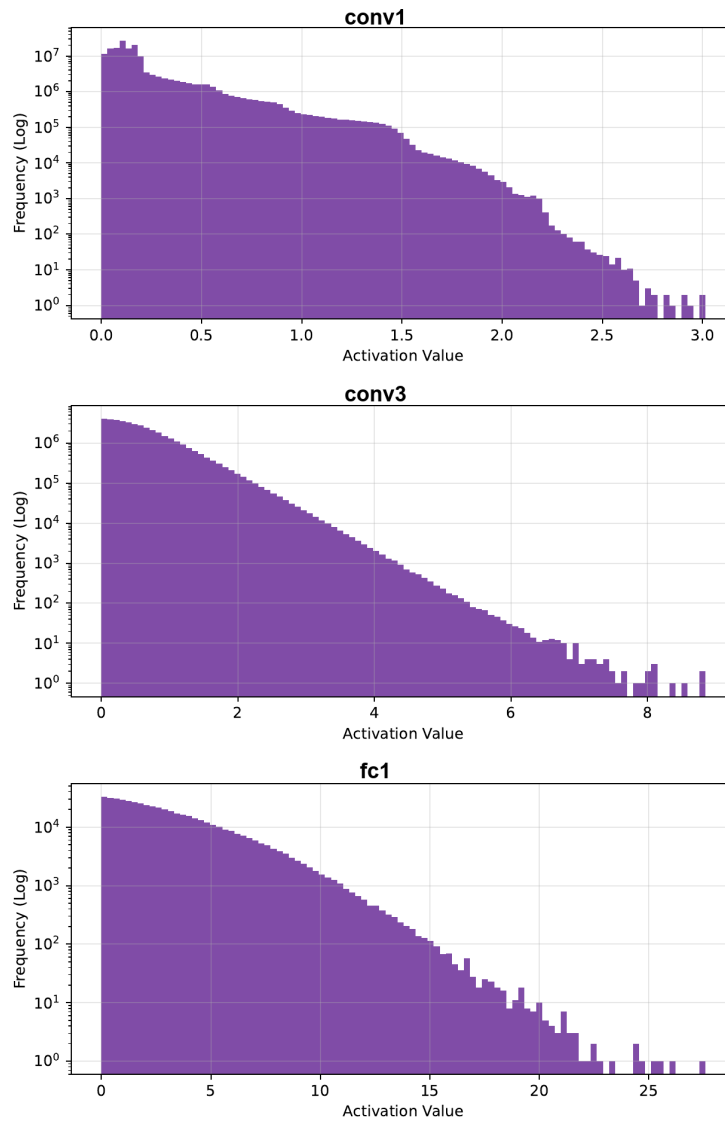


Figure A.12: Distribution of activations per layer (`conv1`, `conv3` and `fc1`), on the validation set of *FashionMNIST*, with regularization method BatchNorm and parameter 1.0.

Dropout = 0.6 | Dataset: Fashion-MNIST

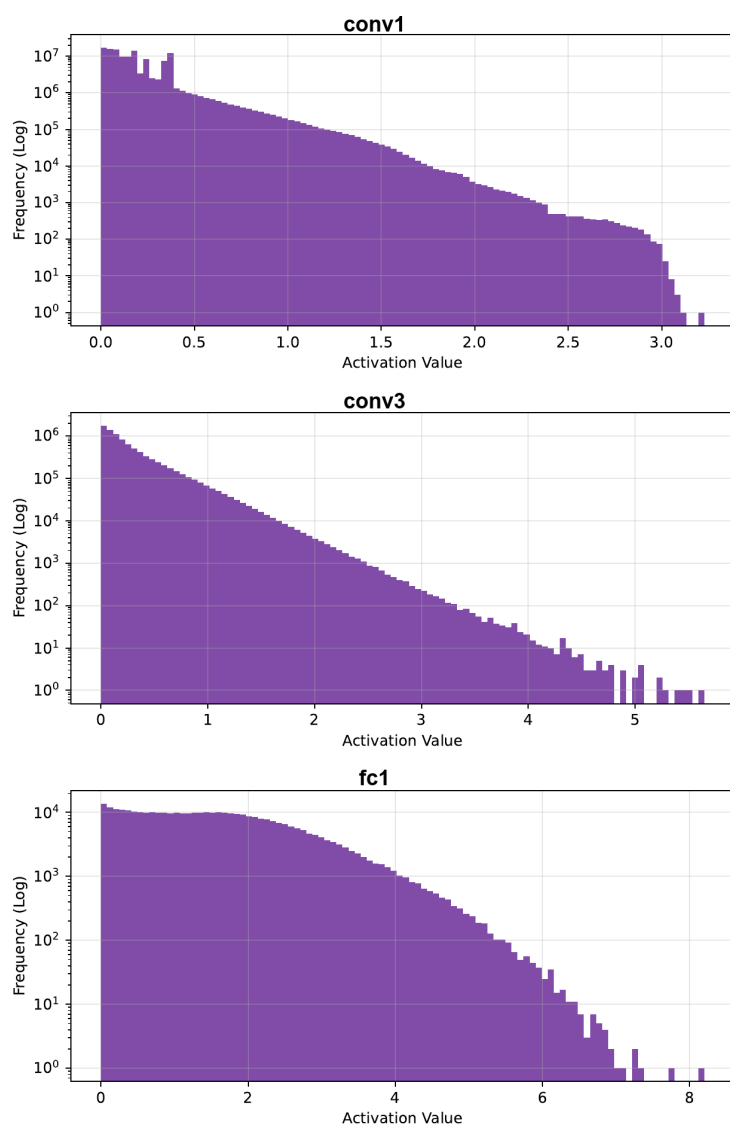


Figure A.13: Distribution of activations per layer (`conv1`, `conv3` and `fc1`), on the validation set of *FashionMNIST*, with regularization method Dropout and parameter 0.6.

GaussianNoise = 0.2 | Dataset: Fashion-MNIST

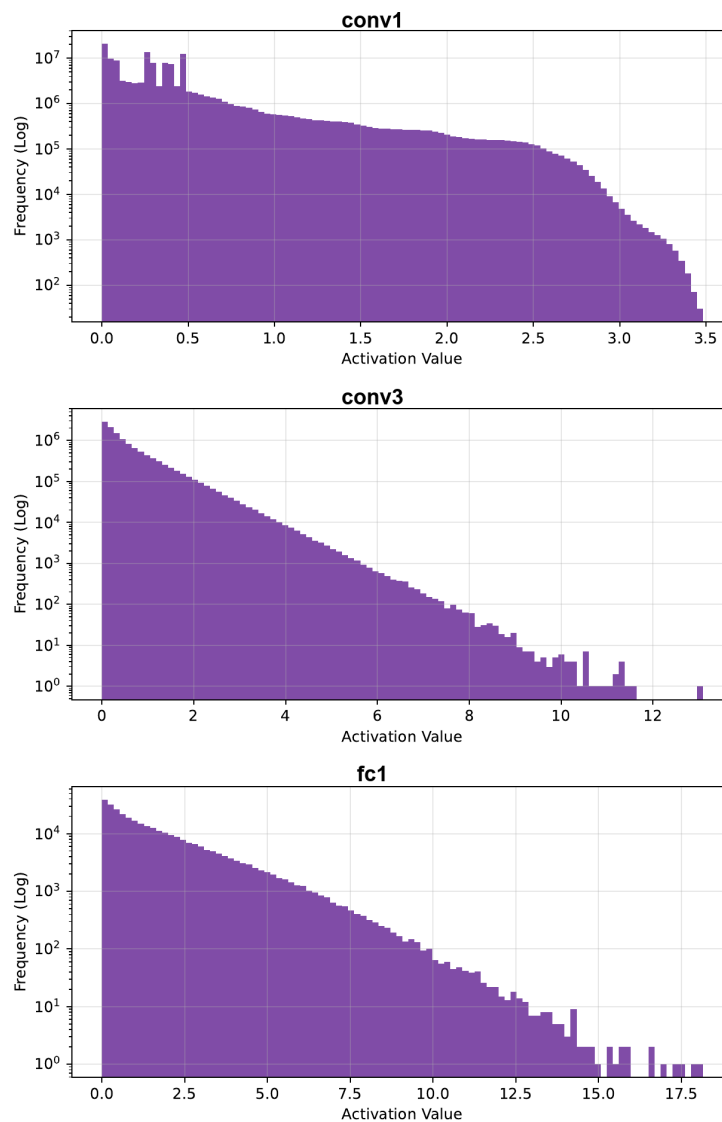


Figure A.14: Distribution of activations per layer (`conv1`, `conv3` and `fc1`), on the validation set of *FashionMNIST*, with regularization method GaussianNoise and parameter 0.2.

CORRELATIONS BETWEEN CONSIDERED INTERNAL METRICS

Comparación de covarianza entre métricas internas - CIFAR10

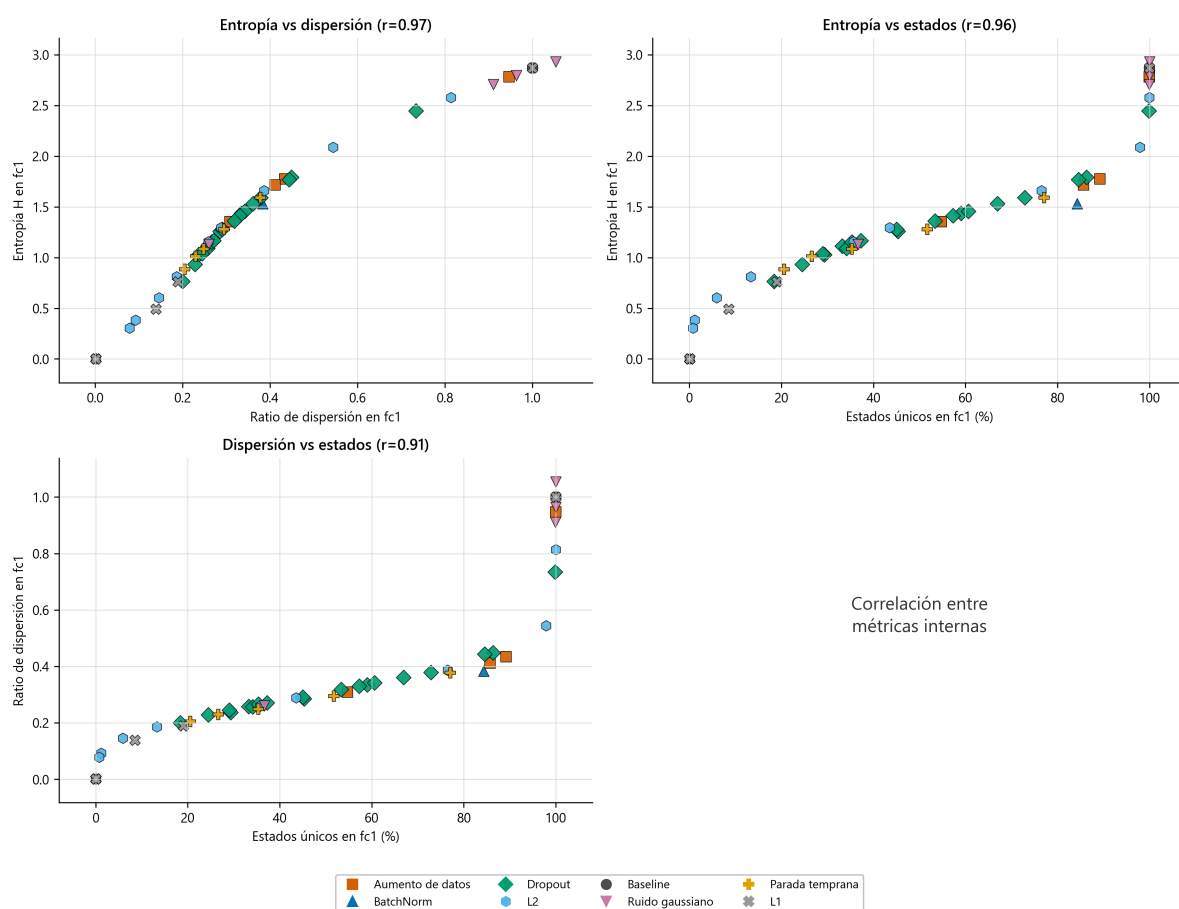


Figure B.1: Relationship between entropy, dispersion ratio and percentage of unique states in $fc1$ for CIFAR-10. The bottom right panel summarizes the comparison between internal metrics.

Métricas internas frente a accuracy de validación con zoom - CIFAR10

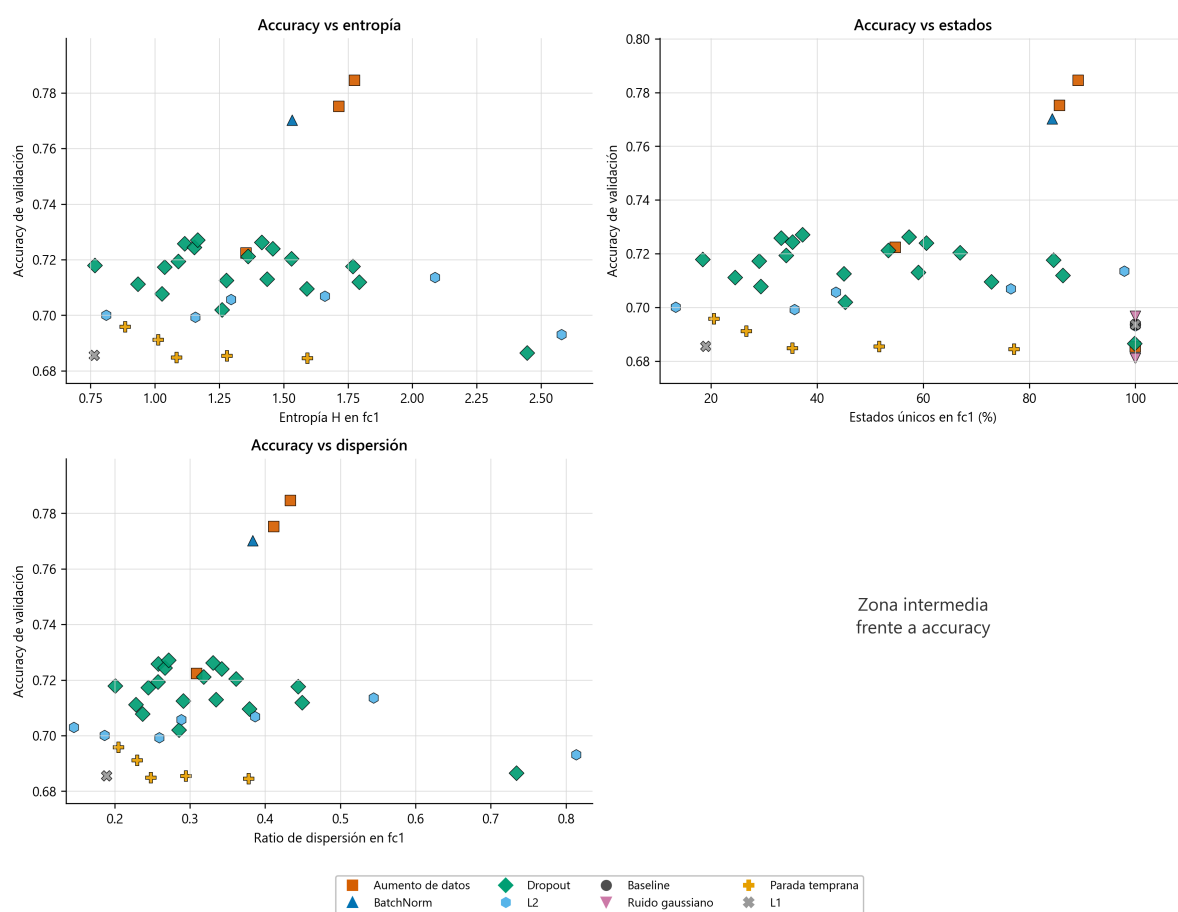


Figure B.2: Relationship between entropy, dispersion ratio and percentage of unique states in $fc1$ with validation accuracy on CIFAR-10. The axes are zoomed in to highlight the intermediate higher-performance region.

Comparación de covarianza entre métricas internas - SVHN

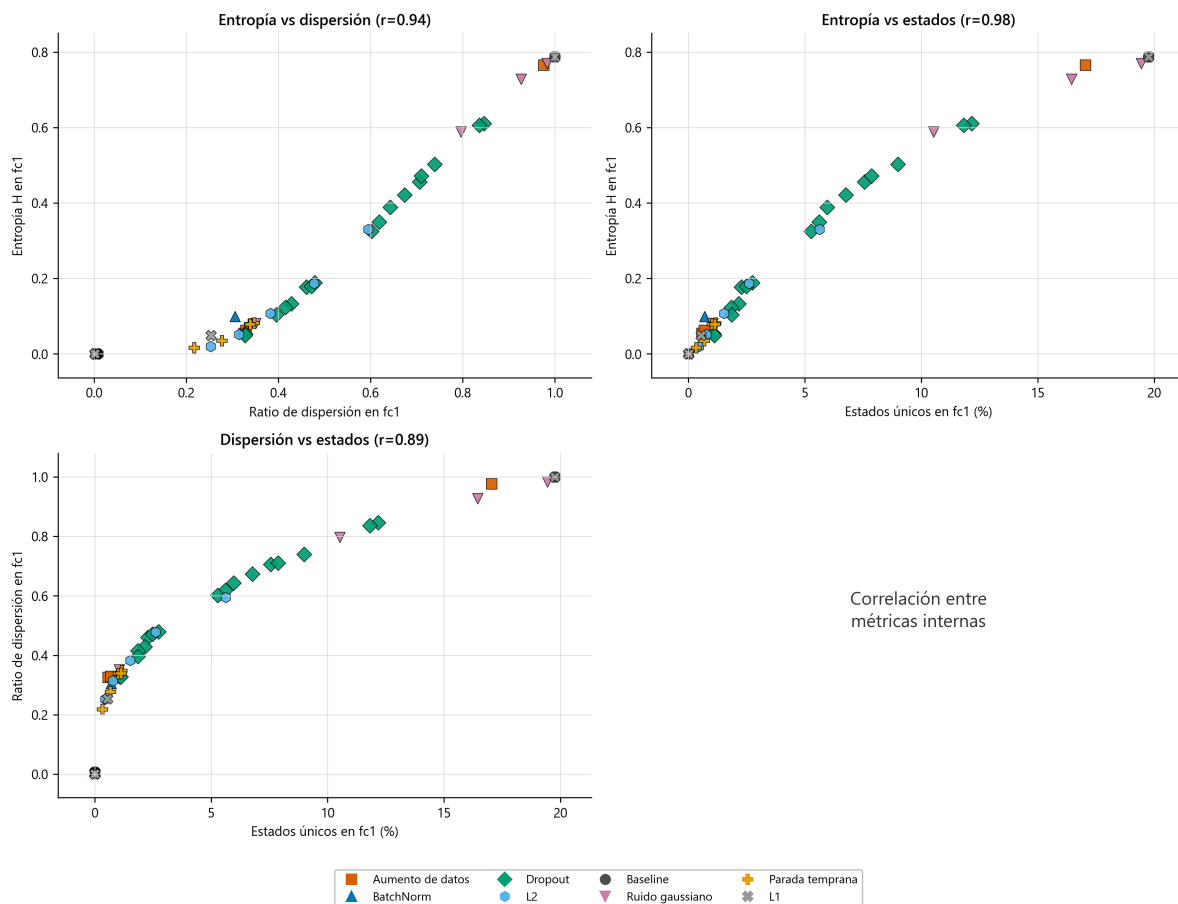


Figure B.3: Relationship between entropy, dispersion ratio and percentage of unique states in $fc1$ for SVHN. The bottom right panel summarizes the comparison between internal metrics.

Métricas internas frente a accuracy de validación con zoom - SVHN

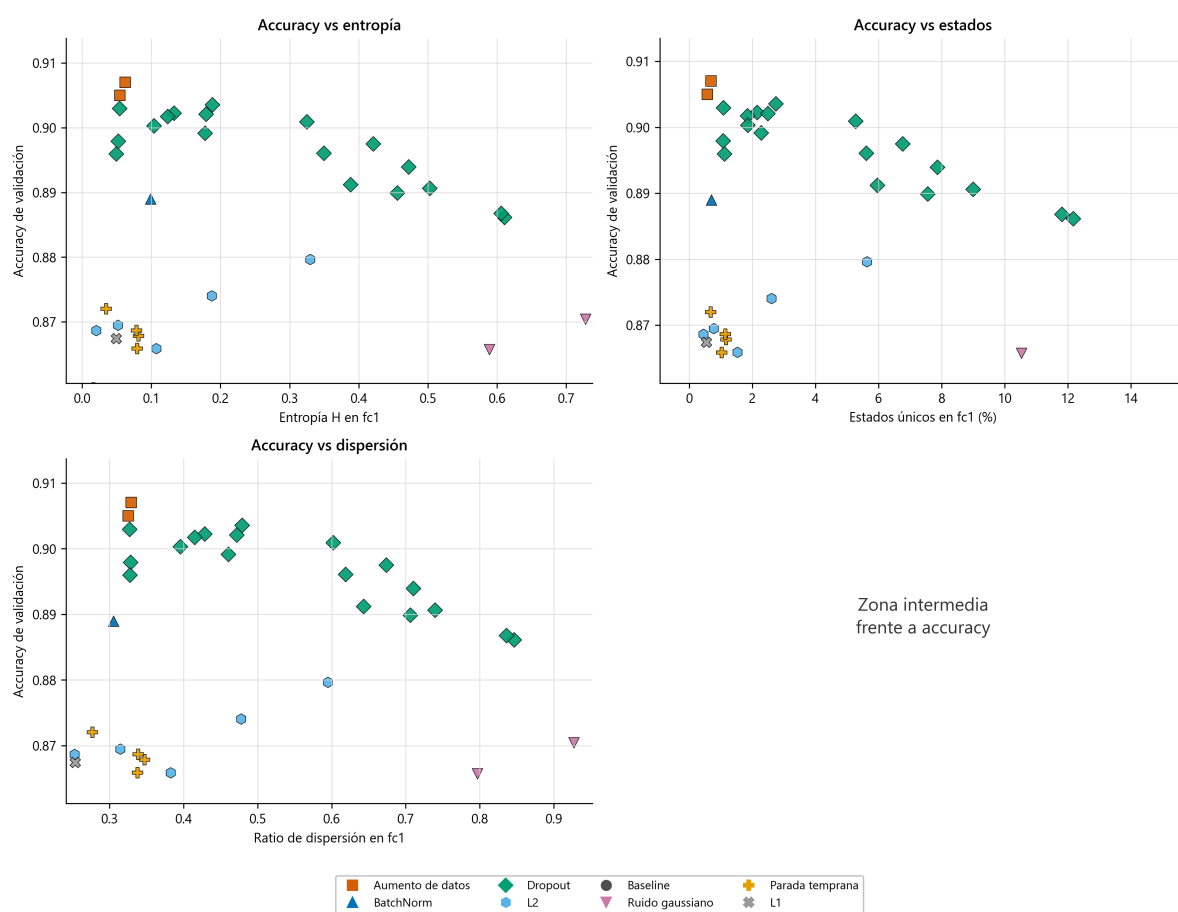


Figure B.4: Relationship between entropy, dispersion ratio and percentage of unique states in $fc1$ with validation accuracy on SVHN. The axes are zoomed in to highlight the intermediate higher-performance region.

Comparación de covarianza entre métricas internas - CIFAR100

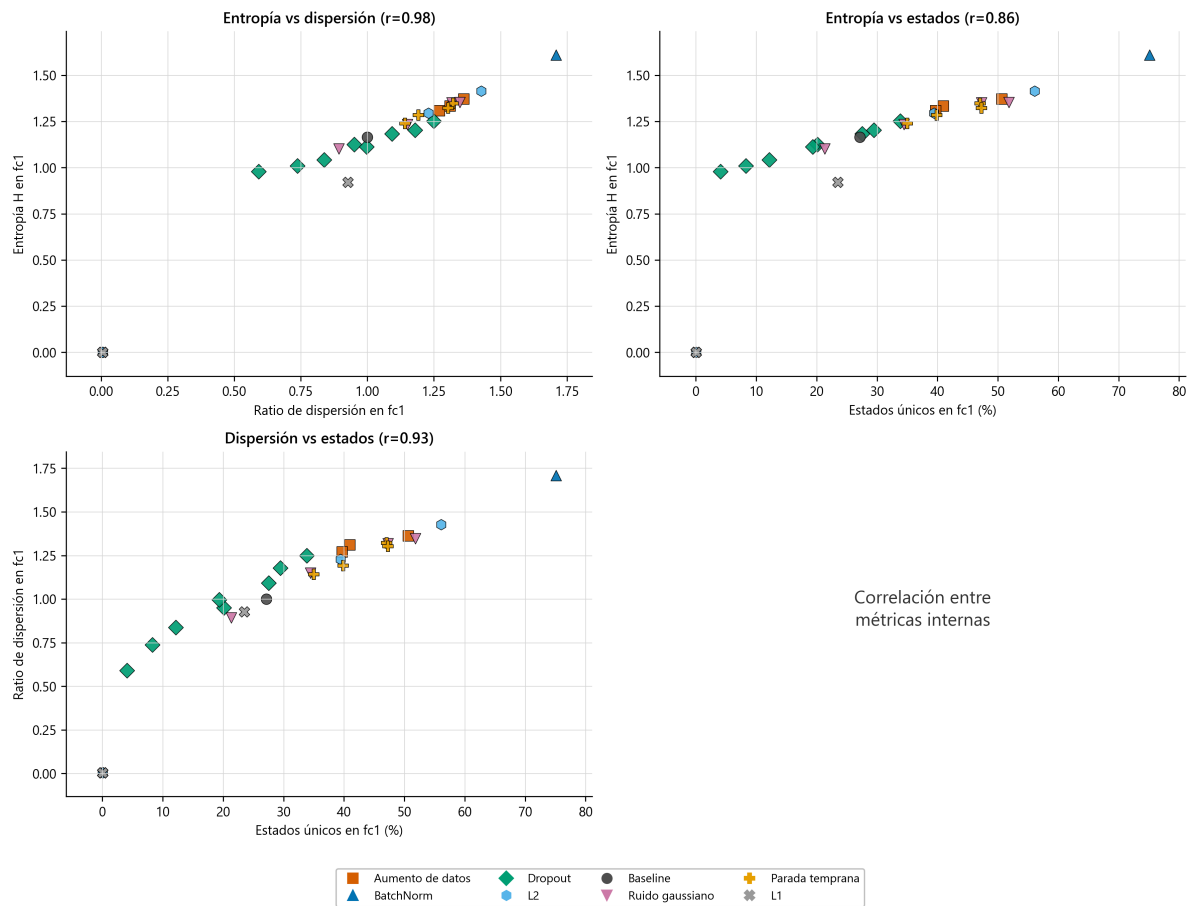


Figure B.5: Relationship between entropy, dispersion ratio and percentage of unique states in $fc1$ for CIFAR-100. The bottom right panel summarizes the comparison between internal metrics.

Métricas internas frente a accuracy de validación con zoom - CIFAR100

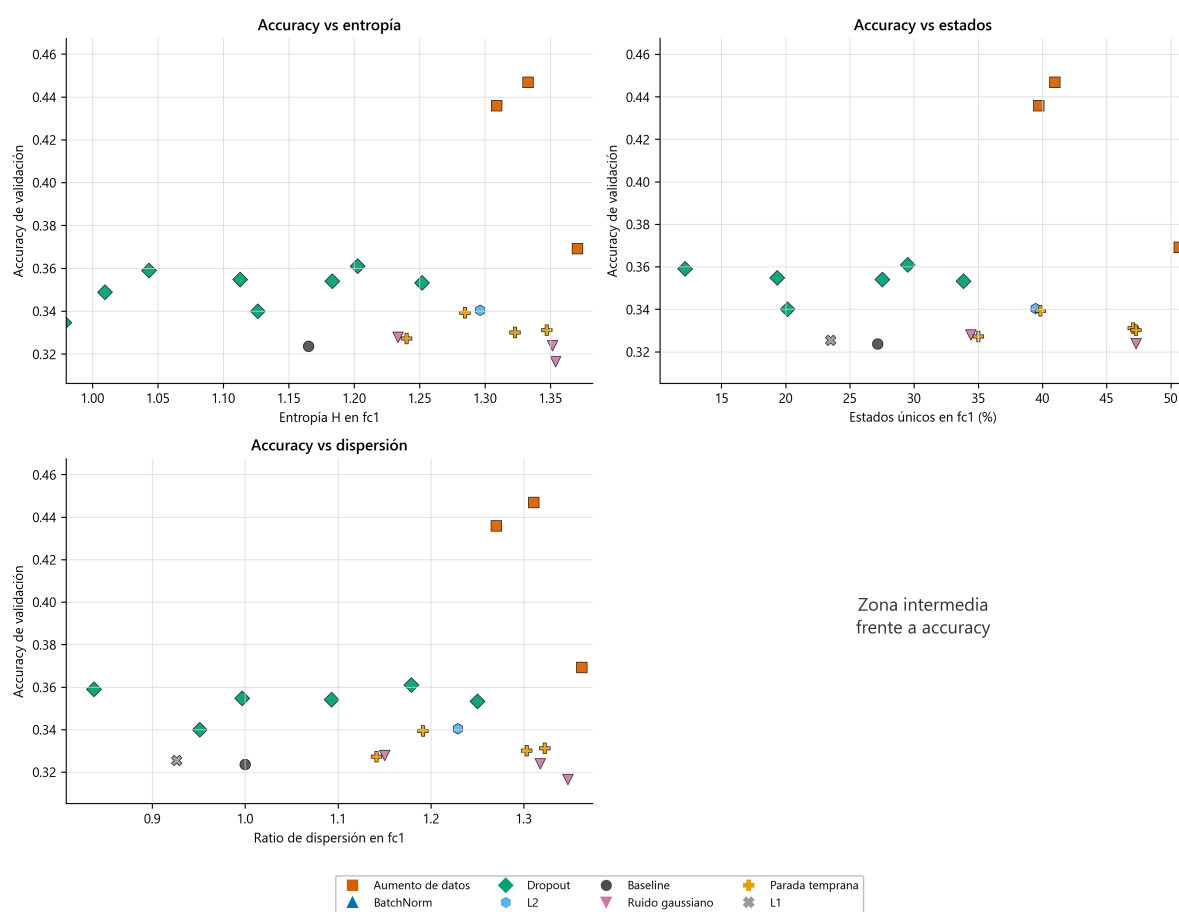


Figure B.6: Relationship between entropy, dispersion ratio and percentage of unique states in $fc1$ with validation accuracy on CIFAR-100. The axes are zoomed in to highlight the intermediate higher-performance region.

Comparación de covarianza entre métricas internas - FashionMNIST

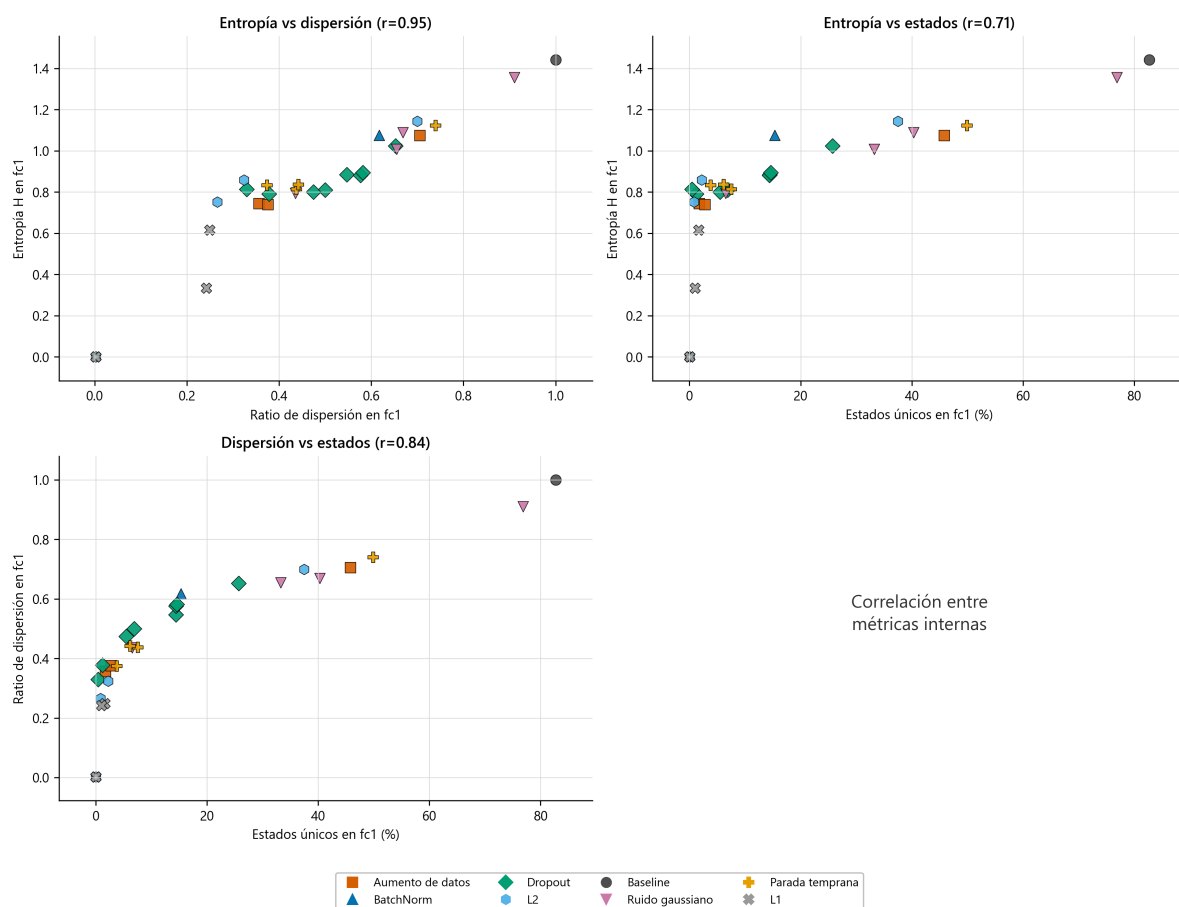


Figure B.7: Relationship between entropy, dispersion ratio and percentage of unique states in $fc1$ for Fashion-MNIST. The bottom right panel summarizes the comparison between internal metrics.

Métricas internas frente a accuracy de validación con zoom - FashionMNIST

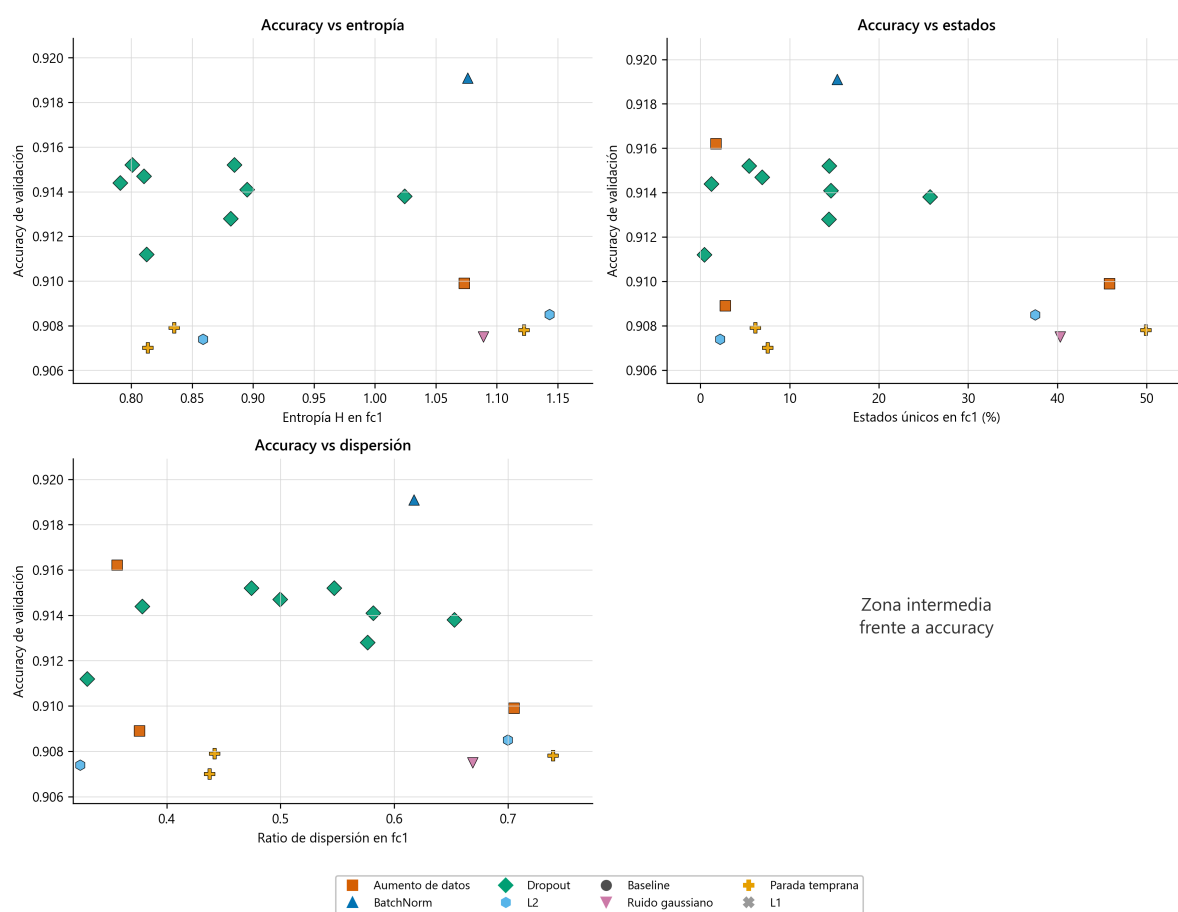


Figure B.8: Relationship between entropy, dispersion ratio and percentage of unique states in $fc1$ with validation accuracy on Fashion-MNIST. The axes are zoomed in to highlight the intermediate higher-performance region.

ACCURACY RETENTION TABLES BY DATA NOISE LEVEL

Method	Acc. $\sigma=0$	Acc. $\sigma=0.05$	Retention (%)
CIFAR-10			
Gaussian noise	69.630	68.820	98.8
L1	65.380	64.517	98.7
L2	71.250	70.307	98.7
Baseline	68.530	67.280	98.2
Early stopping	69.6	67.5	97.0
Data augmentation	77.680	73.783	95.0
Dropout	71.760	68.087	94.9
BatchNorm	76.770	70.010	91.2
SVHN			
L2	87.953	86.951	98.9
Baseline	87.980	86.801	98.7
Gaussian noise	87.827	86.583	98.6
Early stopping	87.0	85.7	98.5
Dropout	90.208	88.490	98.1
L1	84.784	82.711	97.6
BatchNorm	89.213	86.465	96.9
Data augmentation	89.947	87.067	96.8
CIFAR-100			
Gaussian noise	28.320	27.700	97.8
L2	32.920	32.077	97.4
Dropout	33.690	32.730	97.2
Early stopping	33.9	32.9	96.9
Baseline	27.040	26.150	96.7
L1	27.240	25.963	95.3
Data augmentation	42.800	36.907	86.2
BatchNorm	41.610	34.187	82.2
FashionMNIST			
L2	90.790	90.517	99.7
Data augmentation	91.340	91.053	99.7
Gaussian noise	91.100	90.710	99.6
Baseline	91.100	90.683	99.5
L1	89.260	88.847	99.5
Early stopping	90.8	90.3	99.5
Dropout	91.180	90.687	99.5
BatchNorm	91.870	90.960	99.0

Table C.1: Accuracy retention against Gaussian noise in data ($\sigma=0.05$) per method on the four evaluated datasets.

Method	Acc. $\sigma=0$	Acc. $\sigma=0.1$	Retention (%)
CIFAR-10			
Gaussian noise	69.630	65.573	94.2
L1	65.380	60.890	93.1
L2	71.250	65.653	92.1
Baseline	68.530	63.073	92.0
Early stopping	69.6	60.9	87.5
Data augmentation	77.680	62.027	79.8
Dropout	71.760	56.750	79.1
BatchNorm	76.770	49.273	64.2
SVHN			
L2	87.953	84.000	95.5
Baseline	87.980	82.972	94.3
Gaussian noise	87.827	82.757	94.2
Early stopping	87.0	81.1	93.2
Dropout	90.208	82.282	91.2
L1	84.784	76.476	90.2
Data augmentation	89.947	75.917	84.4
BatchNorm	89.213	75.051	84.1
CIFAR-100			
L2	32.920	29.170	88.6
Gaussian noise	28.320	25.050	88.5
Baseline	27.040	23.220	85.9
Early stopping	33.9	28.7	84.7
Dropout	33.690	27.787	82.5
L1	27.240	20.967	77.0
Data augmentation	42.800	21.327	49.8
BatchNorm	41.610	20.187	48.5
FashionMNIST			
L1	89.260	88.123	98.7
L2	90.790	89.573	98.7
Gaussian noise	91.100	89.680	98.4
Baseline	91.100	89.563	98.3
Early stopping	90.8	89.1	98.2
Dropout	91.180	89.303	97.9
Data augmentation	91.340	88.670	97.1
BatchNorm	91.870	88.117	95.9

Table C.2: Accuracy retention against Gaussian noise in data ($\sigma=0.1$) per method on the four evaluated datasets.

Method	Acc. $\sigma=0$	Acc. $\sigma=0.2$	Retention (%)
CIFAR-10			
Gaussian noise	69.630	53.037	76.2
L1	65.380	46.910	71.7
Baseline	68.530	48.853	71.3
L2	71.250	49.180	69.0
Early stopping	69.6	44.1	63.4
Data augmentation	77.680	38.587	49.7
Dropout	71.760	34.890	48.6
BatchNorm	76.770	26.770	34.9
SVHN			
L2	87.953	72.023	81.9
Baseline	87.980	69.626	79.1
Gaussian noise	87.827	68.983	78.5
Early stopping	87.0	66.4	76.3
Dropout	90.208	64.061	71.0
L1	84.784	59.413	70.1
BatchNorm	89.213	48.353	54.2
Data augmentation	89.947	47.545	52.9
CIFAR-100			
L2	32.920	18.300	55.6
Gaussian noise	28.320	15.140	53.5
Baseline	27.040	14.090	52.1
Early stopping	33.9	16.6	49.1
Dropout	33.690	14.623	43.4
L1	27.240	10.160	37.3
BatchNorm	41.610	7.940	19.1
Data augmentation	42.800	7.440	17.4
FashionMNIST			
L2	90.790	86.390	95.2
Baseline	91.100	86.383	94.8
Gaussian noise	91.100	86.257	94.7
Early stopping	90.8	85.4	94.1
L1	89.260	83.857	93.9
Dropout	91.180	84.483	92.7
BatchNorm	91.870	75.373	82.0
Data augmentation	91.340	60.113	65.8

Table C.3: Accuracy retention against Gaussian noise in data ($\sigma=0.2$) per method on the four evaluated datasets.

Method	Acc. $\sigma=0$	Acc. $\sigma=0.3$	Retention (%)
CIFAR-10			
Gaussian noise	69.630	39.143	56.2
Baseline	68.530	36.097	52.7
L1	65.380	34.033	52.1
L2	71.250	32.520	45.6
Early stopping	69.6	31.7	45.6
Dropout	71.760	23.277	32.4
Data augmentation	77.680	24.460	31.5
BatchNorm	76.770	20.073	26.1
SVHN			
L2	87.953	58.483	66.5
Baseline	87.980	54.722	62.2
Gaussian noise	87.827	53.813	61.3
Early stopping	87.0	51.3	58.9
Dropout	90.208	47.684	52.9
L1	84.784	44.356	52.3
BatchNorm	89.213	32.810	36.8
Data augmentation	89.947	29.089	32.3
CIFAR-100			
L2	32.920	10.273	31.2
Gaussian noise	28.320	8.410	29.7
Baseline	27.040	7.547	27.9
Early stopping	33.9	8.8	26.0
Dropout	33.690	7.587	22.5
L1	27.240	5.837	21.4
Data augmentation	42.800	4.083	9.5
BatchNorm	41.610	3.813	9.2
FashionMNIST			
Gaussian noise	91.100	80.577	88.4
L2	90.790	80.147	88.3
Baseline	91.100	80.113	87.9
Early stopping	90.8	78.1	86.0
L1	89.260	74.937	84.0
Dropout	91.180	75.243	82.5
BatchNorm	91.870	51.183	55.7
Data augmentation	91.340	31.727	34.7

Table C.4: Accuracy retention against Gaussian noise in data ($\sigma=0.3$) per method on the four evaluated datasets.

Method	Acc. $\sigma=0$	Acc. $\sigma=0.4$	Retention (%)
CIFAR-10			
Baseline	68.530	27.750	40.5
Gaussian noise	69.630	27.760	39.9
L1	65.380	25.833	39.5
Early stopping	69.6	24.5	35.2
L2	71.250	22.497	31.6
Dropout	71.760	18.293	25.5
Data augmentation	77.680	17.927	23.1
BatchNorm	76.770	17.487	22.8
SVHN			
L2	87.953	46.623	53.0
Baseline	87.980	42.569	48.4
Gaussian noise	87.827	41.694	47.5
Early stopping	87.0	39.6	45.5
Dropout	90.208	36.132	40.1
L1	84.784	33.308	39.3
BatchNorm	89.213	25.490	28.6
Data augmentation	89.947	20.162	22.4
CIFAR-100			
L2	32.920	6.190	18.8
Gaussian noise	28.320	4.993	17.6
Baseline	27.040	4.577	16.9
Early stopping	33.9	5.3	15.6
L1	27.240	3.700	13.6
Dropout	33.690	4.423	13.1
Data augmentation	42.800	2.610	6.1
BatchNorm	41.610	2.413	5.8
FashionMNIST			
Baseline	91.100	71.130	78.1
Gaussian noise	91.100	70.300	77.2
L2	90.790	67.257	74.1
Early stopping	90.8	66.5	73.2
L1	89.260	59.480	66.6
Dropout	91.180	58.600	64.3
BatchNorm	91.870	31.713	34.5
Data augmentation	91.340	23.963	26.2

Table C.5: Accuracy retention against Gaussian noise in data ($\sigma=0.4$) per method on the four evaluated datasets.

Method	Acc. $\sigma=0$	Acc. $\sigma=0.5$	Retention (%)
CIFAR-10			
Baseline	68.530	22.793	33.3
L1	65.380	20.957	32.1
Gaussian noise	69.630	20.787	29.9
Early stopping	69.6	20.6	29.6
L2	71.250	17.773	24.9
Dropout	71.760	16.463	22.9
BatchNorm	76.770	16.103	21.0
Data augmentation	77.680	14.517	18.7
SVHN			
L2	87.953	37.017	42.1
Baseline	87.980	33.561	38.1
Gaussian noise	87.827	33.278	37.9
Early stopping	87.0	31.1	35.8
Dropout	90.208	28.352	31.4
L1	84.784	26.636	31.4
BatchNorm	89.213	21.789	24.4
Data augmentation	89.947	15.270	17.0
CIFAR-100			
L2	32.920	4.047	12.3
Gaussian noise	28.320	3.413	12.1
Baseline	27.040	3.253	12.0
Early stopping	33.9	3.7	10.8
L1	27.240	2.673	9.8
Dropout	33.690	2.883	8.6
Data augmentation	42.800	2.010	4.7
BatchNorm	41.610	1.917	4.6
FashionMNIST			
Baseline	91.100	60.280	66.2
Gaussian noise	91.100	56.913	62.5
Early stopping	90.8	53.7	59.1
L2	90.790	50.177	55.3
L1	89.260	41.880	46.9
Dropout	91.180	41.957	46.0
BatchNorm	91.870	20.093	21.9
Data augmentation	91.340	19.633	21.5

Table C.6: Accuracy retention against Gaussian noise in data ($\sigma=0.5$) per method on the four evaluated datasets.

ACCURACY RETENTION TABLES BY WEIGHT NOISE LEVEL

Method	Ret. conv1 (%)	Ret. conv3 (%)	Ret. fc1 (%)
CIFAR-10			
Gaussian noise	100.1	100.1	99.9
Data augmentation	99.9	99.9	99.9
L2	100.0	100.0	99.8
Early stopping	99.8	100.0	99.9
Dropout	99.8	100.2	100.0
BatchNorm	99.8	100.0	99.8
Baseline	99.8	99.9	99.8
L1	99.9	100.0	99.8
SVHN			
L2	100.0	100.1	100.0
Data augmentation	100.1	100.0	100.0
L1	100.0	100.0	100.1
Dropout	100.0	100.0	100.0
Early stopping	100.0	100.0	99.9
Baseline	100.0	99.9	99.9
BatchNorm	100.0	99.9	99.9
Gaussian noise	100.0	100.0	99.9
CIFAR-100			
L1	100.3	100.5	100.9
L2	100.1	100.3	100.1
Gaussian noise	100.2	100.2	100.1
Early stopping	99.8	99.9	99.8
Dropout	99.7	99.7	100.0
Data augmentation	100.1	99.8	99.6
BatchNorm	99.7	99.5	99.8
Baseline	99.9	100.0	99.4
FashionMNIST			
Dropout	100.0	100.0	100.0
Early stopping	100.0	100.0	100.0
Baseline	100.0	100.0	100.0
Gaussian noise	100.0	100.0	100.0
L2	100.0	100.0	100.0
Data augmentation	100.0	100.0	99.9
L1	100.0	99.9	100.0
BatchNorm	100.0	99.9	99.9

Table D.1: Accuracy retention against Gaussian noise in weights ($\sigma=0.005$) per layer and method on the four evaluated datasets.

Method	Ret. conv1 (%)	Ret. conv3 (%)	Ret. fc1 (%)
CIFAR-10			
Early stopping	99.8	99.8	99.8
L1	99.9	99.9	99.8
Gaussian noise	100.2	100.1	99.8
Dropout	99.7	99.9	99.9
Baseline	99.9	99.7	99.7
Data augmentation	99.8	99.6	99.7
L2	100.0	100.2	99.5
BatchNorm	99.7	99.3	99.8
SVHN			
Data augmentation	100.0	100.0	99.9
Dropout	99.9	99.9	99.9
Early stopping	100.0	100.0	99.9
Baseline	100.0	100.0	99.9
L1	100.0	100.1	99.8
Gaussian noise	100.0	100.0	99.8
L2	100.0	99.9	99.7
BatchNorm	99.8	99.8	99.7
CIFAR-100			
L1	100.2	100.9	99.9
Dropout	99.8	99.9	99.7
Gaussian noise	100.0	100.0	99.7
Early stopping	99.8	99.9	99.3
L2	100.0	100.1	99.3
Data augmentation	99.5	99.6	99.2
BatchNorm	99.7	99.0	99.3
Baseline	99.8	99.9	98.9
FashionMNIST			
Dropout	100.0	100.0	100.1
Early stopping	100.0	100.0	100.0
Gaussian noise	100.0	100.0	100.0
Data augmentation	100.0	99.9	100.0
Baseline	100.0	100.0	99.9
L2	100.0	100.0	99.9
L1	100.0	100.0	99.9
BatchNorm	100.0	99.8	99.9

Table D.2: Accuracy retention against Gaussian noise in weights ($\sigma=0.01$) per layer and method on the four evaluated datasets.

Method	Ret. conv1 (%)	Ret. conv3 (%)	Ret. fc1 (%)
CIFAR-10			
Dropout	99.6	99.7	99.5
Early stopping	99.7	99.3	99.3
Gaussian noise	99.9	99.4	99.1
Data augmentation	99.6	99.1	99.2
Baseline	99.8	99.0	99.0
L1	99.9	99.3	98.9
BatchNorm	98.1	97.3	98.9
L2	99.8	98.6	96.9
SVHN			
Dropout	99.9	99.8	99.9
Early stopping	100.0	99.8	99.6
Gaussian noise	100.0	99.8	99.4
Data augmentation	99.9	99.9	99.4
Baseline	100.0	99.7	99.2
L1	100.0	99.5	99.0
L2	100.0	99.7	98.5
BatchNorm	99.3	97.9	99.2
CIFAR-100			
Gaussian noise	100.1	99.8	98.9
L1	100.8	99.8	98.6
Baseline	99.8	99.5	98.5
Early stopping	99.8	99.5	98.3
Dropout	99.6	99.4	98.1
Data augmentation	99.3	98.5	97.7
L2	99.8	99.6	97.4
BatchNorm	97.0	97.0	96.8
FashionMNIST			
Dropout	100.0	99.9	99.9
Early stopping	100.0	99.8	99.8
Data augmentation	100.0	99.9	99.8
Gaussian noise	100.0	99.9	99.7
Baseline	100.0	99.7	99.7
L1	100.1	99.8	99.7
L2	100.0	99.8	99.5
BatchNorm	99.4	98.8	99.6

Table D.3: Accuracy retention against Gaussian noise in weights ($\sigma=0.02$) per layer and method on the four evaluated datasets.

Method	Ret. conv1 (%)	Ret. conv3 (%)	Ret. fc1 (%)
CIFAR-10			
Dropout	99.3	99.2	99.4
Early stopping	99.4	99.0	98.7
Data augmentation	99.1	98.4	98.7
Baseline	99.4	98.7	97.8
Gaussian noise	99.9	99.1	97.7
L1	99.3	98.0	96.4
L2	99.6	97.7	94.5
BatchNorm	94.9	94.5	97.3
SVHN			
Dropout	99.9	99.7	99.5
Early stopping	100.0	99.6	99.1
Baseline	100.0	99.5	98.6
Gaussian noise	99.9	99.5	98.4
Data augmentation	100.0	99.5	98.3
L1	100.0	99.5	98.1
L2	100.0	98.8	96.8
BatchNorm	98.2	95.8	98.3
CIFAR-100			
Dropout	99.5	98.5	96.7
Early stopping	99.2	98.8	96.4
Baseline	99.1	99.0	96.2
Gaussian noise	100.1	99.0	95.9
L1	99.3	98.4	95.7
Data augmentation	98.4	96.8	94.9
L2	99.7	98.3	94.2
BatchNorm	90.3	92.8	93.4
FashionMNIST			
Gaussian noise	100.0	99.7	99.4
Dropout	99.9	99.7	99.4
Early stopping	99.9	99.5	99.2
Baseline	99.9	99.4	99.1
Data augmentation	100.0	99.1	99.4
L1	99.9	99.5	99.0
L2	99.9	99.2	98.7
BatchNorm	99.4	97.2	99.1

Table D.4: Accuracy retention against Gaussian noise in weights ($\sigma=0.03$) per layer and method on the four evaluated datasets.

Method	Ret. conv1 (%)	Ret. conv3 (%)	Ret. fc1 (%)
CIFAR-10			
Dropout	99.0	98.2	97.7
Early stopping	99.0	97.6	96.0
Gaussian noise	99.3	96.8	94.6
Data augmentation	98.3	96.0	94.5
Baseline	99.0	97.0	94.0
L1	100.1	95.2	89.7
L2	98.8	93.9	84.5
BatchNorm	88.8	83.9	93.0
SVHN			
Dropout	99.9	99.2	98.8
Early stopping	99.9	99.0	97.4
Gaussian noise	99.9	98.9	95.8
Baseline	99.9	98.8	95.7
Data augmentation	99.7	97.9	94.2
L1	99.9	97.5	94.2
L2	99.9	97.1	88.8
BatchNorm	92.9	87.7	94.7
CIFAR-100			
Gaussian noise	98.9	96.4	91.5
Dropout	98.7	95.8	90.5
Early stopping	99.1	95.8	90.2
Baseline	99.3	95.8	89.8
Data augmentation	97.5	92.8	87.0
L1	98.3	93.6	85.2
L2	99.9	94.8	83.2
BatchNorm	85.5	79.4	84.1
FashionMNIST			
Dropout	99.8	98.8	98.7
Early stopping	99.9	98.5	98.5
Baseline	99.9	98.2	98.3
Data augmentation	99.8	98.1	99.0
Gaussian noise	99.8	98.7	97.9
L2	100.0	97.4	96.8
L1	99.9	98.3	95.1
BatchNorm	97.9	90.0	97.3

Table D.5: Accuracy retention against Gaussian noise in weights ($\sigma=0.05$) per layer and method on the four evaluated datasets.

Method	Ret. conv1 (%)	Ret. conv3 (%)	Ret. fc1 (%)
CIFAR-10			
Dropout	94.4	95.1	96.3
Early stopping	97.0	94.5	92.6
Data augmentation	95.1	91.6	89.1
Baseline	98.7	93.9	88.1
Gaussian noise	98.6	93.4	87.9
L1	100.2	90.2	82.0
L2	96.6	85.6	74.6
BatchNorm	72.2	67.0	86.6
SVHN			
Dropout	99.3	97.8	97.6
Early stopping	99.4	97.5	94.7
Gaussian noise	100.0	96.2	92.3
Baseline	99.5	97.2	91.2
Data augmentation	97.7	93.5	88.0
L1	100.0	94.7	87.3
L2	99.8	93.0	78.9
BatchNorm	82.3	74.2	90.3
CIFAR-100			
Baseline	95.0	93.1	80.9
Early stopping	95.8	91.2	80.6
Dropout	97.1	89.2	80.4
Gaussian noise	97.7	91.1	78.1
Data augmentation	89.0	84.4	76.2
L1	92.4	86.4	71.7
L2	98.3	86.1	69.9
BatchNorm	73.0	59.2	69.1
FashionMNIST			
Dropout	99.3	97.0	97.4
Early stopping	99.4	96.4	96.6
Data augmentation	99.2	95.7	96.0
Baseline	99.4	95.9	95.6
Gaussian noise	100.0	96.1	94.7
L1	99.7	95.7	89.8
L2	99.7	94.5	86.5
BatchNorm	95.4	75.3	92.8

Table D.6: Accuracy retention against Gaussian noise in weights ($\sigma=0.075$) per layer and method on the four evaluated datasets.

Method	Ret. conv1 (%)	Ret. conv3 (%)	Ret. fc1 (%)
CIFAR-10			
Dropout	96.5	91.6	93.2
Early stopping	95.7	89.7	88.2
Gaussian noise	97.5	89.8	82.6
Baseline	95.3	88.0	82.2
Data augmentation	89.0	84.7	82.1
L1	96.6	84.4	73.7
L2	94.7	72.6	59.3
BatchNorm	63.6	58.7	76.9
SVHN			
Dropout	98.9	95.6	95.6
Early stopping	99.3	94.6	91.0
Baseline	99.6	93.7	85.4
Gaussian noise	99.8	94.6	85.0
Data augmentation	98.1	89.5	83.4
L1	99.6	91.3	78.9
L2	99.7	85.9	70.8
BatchNorm	58.9	59.2	83.6
CIFAR-100			
Dropout	94.2	82.4	69.5
Gaussian noise	91.8	85.2	69.4
Early stopping	92.2	85.3	69.4
Baseline	90.9	87.9	69.2
Data augmentation	73.7	72.7	60.3
L1	92.7	78.0	58.8
L2	96.8	78.8	54.5
BatchNorm	50.4	41.6	50.8
FashionMNIST			
Dropout	99.3	93.2	95.2
Early stopping	99.4	92.5	93.1
Baseline	99.4	91.8	90.5
Gaussian noise	99.3	93.4	90.0
Data augmentation	99.5	85.3	94.4
L2	99.7	83.6	84.5
L1	99.1	90.5	82.8
BatchNorm	92.4	66.6	84.0

Table D.7: Accuracy retention against Gaussian noise in weights ($\sigma=0.1$) per layer and method on the four evaluated datasets.

Method	Ret. conv1 (%)	Ret. conv3 (%)	Ret. fc1 (%)
CIFAR-10			
Early stopping	86.8	77.7	77.6
Dropout	83.1	77.5	86.1
Gaussian noise	96.6	75.5	67.7
Baseline	89.2	77.9	67.1
Data augmentation	73.1	64.6	67.0
L1	97.2	65.5	54.2
L2	83.9	51.6	47.6
BatchNorm	51.2	42.0	63.5
SVHN			
Dropout	96.7	88.3	91.6
Early stopping	98.0	88.2	81.4
Gaussian noise	99.6	86.3	72.2
Baseline	98.8	88.2	68.9
L1	99.3	78.6	64.3
Data augmentation	85.0	70.3	61.2
BatchNorm	50.4	48.8	64.6
L2	99.8	68.4	45.9
CIFAR-100			
Baseline	89.5	76.2	50.2
Early stopping	86.0	68.9	49.4
Dropout	80.6	60.9	48.7
Gaussian noise	92.4	71.7	48.3
Data augmentation	58.0	48.1	40.2
L1	76.5	57.6	37.5
L2	90.3	60.3	33.1
BatchNorm	30.8	24.4	35.4
FashionMNIST			
Dropout	96.9	84.4	90.0
Early stopping	98.1	81.8	83.5
Gaussian noise	99.4	79.2	78.7
Baseline	98.9	79.5	75.6
L1	99.4	81.3	73.8
L2	98.9	66.3	67.4
Data augmentation	96.7	57.7	81.3
BatchNorm	82.2	38.9	63.7

Table D.8: Accuracy retention against Gaussian noise in weights ($\sigma=0.15$) per layer and method on the four evaluated datasets.



Universidad Autónoma
de Madrid